

SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis

Evaluating the Effect of Different MPI Implementations on Communication-Computation Overlap in Ocean Modeling

Jintao Lu



SCHOOL OF COMPUTATION, INFORMATION AND TECHNOLOGY

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis

Evaluating the Effect of Different MPI Implementations on Communication-Computation Overlap in Ocean Modeling

Bewertung des Einflusses verschiedener MPI-Implementierungen auf die Überlappung von Kommunikation und Berechnung in der Ozeanmodellierung

Author:	Jintao Lu
Examiner:	Prof. Dr. rer. nat. Martin Schulz
Supervisor:	M.Sc. Anna Mittermair
Submission Date:	14 September 2026

I confirm that this bachelor's thesis is my own work and I have documented all sources and material used.

Munich, 14 September 2026 Jintao Lu

Abstract

Halo exchanges periodically introduce communication overhead in ocean models that use domain decomposition. Nonblocking Message Passing Interface (MPI) operations allow independent computation to be performed between communication initiation and completion, but the degree of communication–computation overlap that can actually be achieved depends on how outstanding communication is progressed.

This thesis investigates how MPI implementation, transport, message size, and progress configuration affect communication–computation overlap in a controlled halo-exchange benchmark.

The Comb benchmark is extended with an adjustable-duration synthetic compute kernel inserted between the initiation and completion of nonblocking communication. Experiments are conducted on the LRZ CoolMUC-4 system, primarily using two nodes with one MPI rank per node and covering 19 message payload sizes. The evaluated configurations include Open MPI over TCP, MPICH over TCP, and Intel MPI over both verbs and TCP. Measurements with and without the injected computation are compared to quantify changes in receive completion time. In addition, a mock backend in Comb that performs no actual network communication is used to estimate processing overhead introduced by the benchmark framework, local memory operations, and timing. Repeated independent runs are used to characterize measurement variability.

The results show that simply using nonblocking MPI communication interfaces does not guarantee effective communication–computation overlap. Without asynchronous progress, Intel MPI exhibits very low measured overlap with both verbs and TCP. Enabling asynchronous progress substantially improves the measured overlap across much of the medium- and large-message range. With progress enabled, Open MPI over TCP shows a sustained decline in measured overlap at larger payload sizes, whereas Intel MPI with both verbs and TCP, as well as MPICH over TCP, maintain relatively high measured overlap over the tested large-message range.

These findings demonstrate that communication–computation overlap must be evaluated for the specific MPI configuration and communication workload. Reductions in receive completion time should be considered together with total runtime and run-to-run variability. The benchmark provides experimental evidence relevant to halo exchanges in ocean models, but does not establish application-level speedups for a complete ocean simulation.

Acknowledgements

I would like to thank my supervisor, M.Sc. Anna Mittermair, for her guidance and support during this thesis. I am also grateful to Prof. Dr. rer. nat. Martin Schulz for the opportunity to carry out this work at the Technical University of Munich.

I gratefully acknowledge the Leibniz Supercomputing Centre (LRZ) for providing access to the CoolMUC-4 system used for the experiments in this thesis. I also thank the developers and maintainers of Comb and the MPI implementations evaluated in this work for making the software underlying this study available.

Contents

Abstract iii

Acknowledgements iv

1. Introduction 1
1.1. Contributions . . . 1

2. Background and Related Work 3
2.1. Communication in Ocean Models . . . 3
2.1.1. Comb Benchmark . . . 3
2.2. MPI Communication Progress and Overlap . . . 4
2.2.1. Blocking and Nonblocking Communication . . . 4
2.2.2. Asynchronous Progress Mechanisms . . . 5
2.2.3. Polling-Based Progress with `MPI_Test` . . . 6
2.3. MPI Implementations and Communication Stacks . . . 7
2.3.1. Intel MPI . . . 8
2.3.2. Open MPI . . . 8
2.3.3. MPICH . . . 8
2.4. Related Work . . . 9

3. Methodology and Experimental Setup 11
3.1. Benchmarks and Code Modifications . . . 11
3.1.1. Comb . . . 11
3.1.2. Injected Computation . . . 11
3.1.3. Placement of the Injected Computation . . . 12
3.2. Hardware and Software Environment . . . 13
3.2.1. CoolMUC-4 . . . 13
3.2.2. Compilers . . . 13
3.2.3. MPI Implementations . . . 13
3.2.4. Build Configuration . . . 14
3.3. MPI Runtime Configuration . . . 14
3.3.1. Communication Stacks and Providers . . . 14
3.3.2. Verbs and TCP . . . 14

3.3.3. Asynchronous Progress Configuration . . . 15
3.3.4. CPU and Rank Binding . . . 16
3.3.5. `wait_all` and `test_all` Completion Variants . . . 16
3.4. Workloads and Experiment Matrix . . . 17
3.4.1. Grid, Ghost Width, and Variables . . . 17
3.4.2. Message Sizes . . . 17
3.4.3. Cycles and Compute Targets . . . 18
3.4.4. Node Count and Process Topology . . . 20
3.5. Timing and Evaluation Metrics . . . 20
3.5.1. Timing Components and Baselines . . . 20
3.5.2. Mock-Corrected Communication Cost . . . 21
3.5.3. Operational Measure of Overlap . . . 22
3.5.4. Statistical Aggregation and Variability . . . 23
3.6. Experimental Procedure . . . 23
3.6.1. Staged Experimental Design . . . 23
3.6.2. Warm-up . . . 24
3.6.3. Repetitions and Paired Runs . . . 24
3.6.4. Execution Order . . . 24
3.6.5. Data Filtering . . . 25
3.7. Reproducibility and Threats to Validity . . . 25

4. Experimental Results and Analysis **26**
4.1. Measurement Validation . . . 26
4.2. Message Size and MPI Implementation . . . 27
4.2.1. Mock-Corrected Communication Cost . . . 27
4.2.2. Overlap across Message Sizes . . . 27
4.2.3. Cross-Implementation Comparison . . . 31
4.2.4. Asynchronous Progress . . . 33
4.2.5. Impact of Completion Strategy: `wait_all` versus `test_all` . . . 38
4.3. Polling During Computation . . . 40
4.3.1. Intel MPI verbs . . . 40
4.3.2. Open MPI TCP . . . 40

5. Conclusion and Future Work **44**
5.1. Summary . . . 44
5.2. Conclusions . . . 44
5.3. Limitations and Future Work . . . 45

A. Reproducibility Information **47**
A.1. Experiment Inventory 47
A.2. Message-Size Overlap Figure Set 47

B. Additional Completion-Strategy Comparisons **52**

List of Figures **56**

List of Tables **58**

Bibliography **59**

1. Introduction

In large-scale ocean models and other grid-based scientific applications, the computational domain is typically divided into multiple subdomains and assigned to different Message Passing Interface (MPI) processes. To ensure the correctness of numerical computations near subdomain boundaries, neighboring processes must periodically exchange halo or ghost-cell data. As the degree of parallelism increases, this neighborhood communication can introduce substantial communication and synchronization overhead. Nonblocking MPI communication makes it possible to perform independent computation while data is being exchanged, and therefore offers an opportunity to hide part of the communication cost through communication–computation overlap. Effective overlap cannot, however, be assumed in every case. Nonblocking communication primarily permits communication and computation to execute concurrently at the logical level, whereas actual overlap depends on the communication progress mechanism provided by the MPI implementation and may vary with message size and the underlying communication mechanism [3].

This thesis investigates these effects through a benchmark-based evaluation on the LRZ CoolMUC-4 system. The LLNL Comb halo-exchange benchmark [8] is extended with an adjustable computation window and explicit polling points. These extensions make it possible to examine how outstanding communication progresses while computation is performed under the selected MPI and transport configurations.

1.1. Contributions

The contributions of this thesis are:

1. A controlled experimental methodology for evaluating communication–computation overlap, implemented using a modified Comb benchmark with an adjustable synthetic computation window and optional polling during computation.

2. A comparative evaluation of Open MPI, MPICH, and Intel MPI on LRZ CoolMUC-4, analyzing how MPI implementation, transport, message size, asynchronous progress, completion strategy, and computation duration influence the measured overlap.

3. A web-based experiment management and visualization tool that supports experiment configuration, script generation, run management, and comparison of measured results. The tool and accompanying experimental materials are provided through the repository linked in section A.1.

The results characterize communication–computation overlap under the evaluated benchmark, platform, and runtime configurations. They should therefore be interpreted as evidence for the tested environment rather than as direct evidence of application-level performance in a complete ocean model.

The remainder of this thesis presents the background and related work in chapter 2, the benchmark methodology and experimental setup in chapter 3, and the measurements and analysis in chapter 4. The final chapter summarizes the conclusions and directions for future work.

2. Background and Related Work

2.1. Communication in Ocean Models

Large-scale ocean models commonly discretize the computational domain on structured grids and parallelize the resulting workload across multiple processing elements [13]. Numerical operations near subdomain boundaries require data from neighboring subdomains, giving rise to halo exchanges [12, 13].

Since part of the computation on interior grid points does not depend on newly received boundary data, halo exchange can be initiated before independent computation is performed [12, 13]. Nonblocking MPI provides the corresponding initiation–completion structure and therefore creates an opportunity for communication–computation overlap [11, Sec. 4.7]. In this study, Comb is used as the primary benchmark for constructing and evaluating this type of structured-grid communication behavior [8]. The study-specific modifications and experimental configuration are described in Chapter 3.

2.1.1. Comb Benchmark

Comb is a communication-performance benchmark developed at Lawrence Livermore National Laboratory for studying communication patterns on high-performance computing systems. Its central workload is a configurable structured-mesh halo exchange, in which a global multidimensional grid is decomposed among parallel processes and boundary data are exchanged between neighboring subdomains [8]. Comb was designed to explore the performance trade-offs between communication methods, execution policies, and memory spaces, and supports combinations of sequential, OpenMP, and accelerator-oriented execution strategies [8].

For structured-grid communication, Comb constructs halo regions at the faces, edges, and corners of each local subdomain according to the selected ghost width. Data destined for the same neighboring MPI rank are aggregated into a single communication buffer before transmission. Consequently, the message size observed by the MPI implementation represents the aggregated payload associated with a communication partner rather than the data volume of an individual halo face [8, `include/CommFactory.hpp`]. This communication structure resembles the

nearest-neighbor exchange pattern commonly found in domain-decomposed scientific applications, including ocean and atmospheric models [12].

A communication cycle separates message initiation from completion. Nonblocking receives are posted first, followed by packing and posting of nonblocking sends; receive and send requests are completed in later phases [8, `include/do_cycles.hpp`]. This separation makes Comb particularly suitable for investigating communication–computation overlap, since outstanding MPI communication exists between the posting and completion phases. Comb therefore provides the structured and configurable communication pattern used as the primary experimental benchmark in this study. The study-specific modifications and timing methodology are described in Chapter 3.

2.2. MPI Communication Progress and Overlap

Nonblocking MPI communication separates the initiation of a communication operation from its completion and therefore creates the possibility of performing useful computation while communication is outstanding. However, this logical concurrency does not by itself guarantee that data transfer continues during the computation interval. Actual communication–computation overlap depends on how the MPI implementation makes progress on pending operations [3].

This section first distinguishes blocking and nonblocking communication and explains their relationship to communication–computation overlap. It then discusses two mechanisms considered in this thesis for advancing nonblocking communication during computation: implementation-provided asynchronous progress and application-driven polling with `MPI_Test` [19, 11, 14].

2.2.1. Blocking and Nonblocking Communication

MPI point-to-point communication can be expressed using either blocking or nonblocking operations. Blocking calls such as `MPI_Send` and `MPI_Recv` return only after their respective completion conditions have been satisfied. For example, completion of a blocking send guarantees that the application may safely reuse the send buffer, but does not necessarily imply that the corresponding receive operation has already completed [11, Sec. 4.2].

Nonblocking operations separate communication initiation from completion. Calls such as `MPI_Isend` and `MPI_Irecv` initiate the operation and return an `MPI_Request` object, allowing the application to continue executing while the communication remains outstanding. The request is subsequently completed or tested using routines such as `MPI_Wait`, `MPI_Test`, or their multiple-request variants such as `MPI_Waitall` and `MPI_Testall` [11, Secs. 4.7.3–4.7.5].

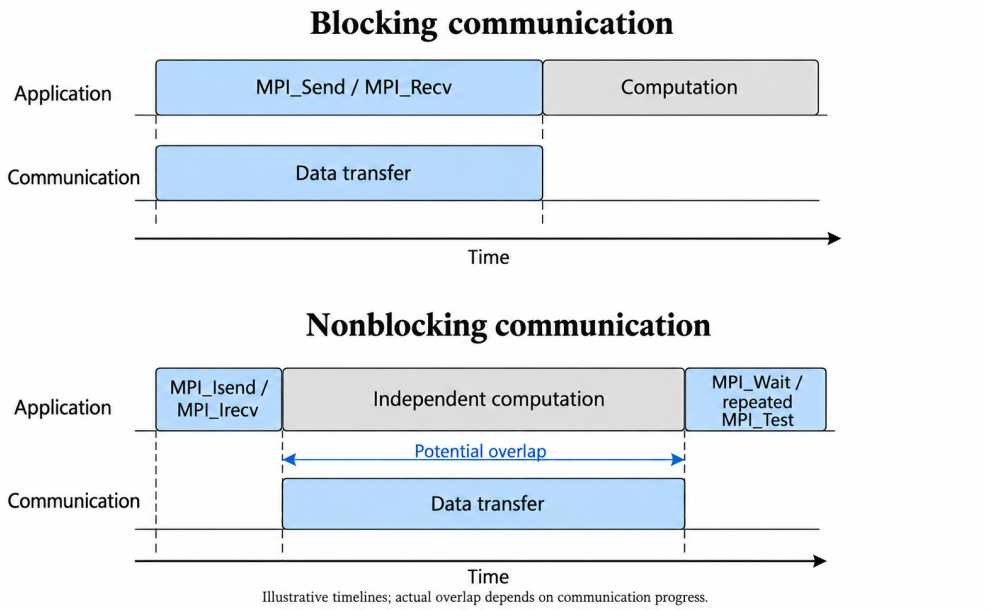


Figure 2.1.: Schematic comparison of blocking and nonblocking MPI point-to-point communication. Nonblocking operations separate initiation from completion, allowing independent computation to potentially overlap with communication. Actual overlap depends on communication progress during computation. The timelines are illustrative and not to scale.

As illustrated in Figure 2.1, this separation between initiation and completion creates a potential interval in which independent computation can be performed while communication is outstanding. Nonblocking communication therefore provides the programming structure required for communication–computation overlap. However, it does not by itself guarantee that communication actually progresses while the application is performing computation without invoking communication or completion routines; this depends on the progress semantics and progress mechanisms of the MPI implementation [3].

2.2.2. Asynchronous Progress Mechanisms

Asynchronous progress typically applies to nonblocking communication operations that have been initiated but not yet completed. For example, after communication has been initiated through `MPI_Isend` and `MPI_Irecv`, the corresponding requests may remain incomplete while the application thread continues executing computa-

tion [11, Sec. 4.7]. Progress on these communication operations can either be assisted explicitly by the application or provided independently by the MPI implementation. Asynchronous progress refers to the latter case and aims to decouple communication progress from application-level polling so that communication can continue to advance while the application thread performs computation [19].

A common software-based mechanism is the use of one or more dedicated progress threads. These threads execute independently of the application thread and can advance communication operations that have not yet completed while application computation continues. Depending on the MPI implementation, their responsibilities may include request processing, protocol progression, and interaction with the underlying communication layer [19]. Asynchronous progress can also be partially supported by network hardware, for example through communication offload to the network interface, although the available mechanisms depend on the MPI implementation and its underlying communication stack [11, Sec. 3.9].

Software progress threads consume processor resources, and their performance benefit therefore depends on CPU-resource allocation. A study of MPI nonblocking collectives on manycore processors has shown that progress-thread placement and the use of dedicated processor cores can affect the performance benefit obtained from communication–computation overlap [2]. If a progress thread shares processor resources with an application thread, additional resource contention and runtime overhead may be introduced [19]. Assigning dedicated CPU resources to progress can reduce such contention, but it also reduces the processor resources available for application computation. Consequently, enabling asynchronous progress does not necessarily improve overall performance; its effectiveness depends on whether the additional communication progress obtained during computation outweighs the corresponding resource consumption and runtime overhead [19].

2.2.3. Polling-Based Progress with `MPI_Test`

Another way to facilitate the progress of nonblocking communication is to periodically invoke nonblocking completion-test routines during computation. Functions such as `MPI_Test` and `MPI_Testall` check whether one or more communication requests have completed. If the corresponding operations are still incomplete, these routines do not block in the same way as `MPI_Wait`, but instead return the current completion status. The application can therefore continue computation between polling points while periodically checking the state of the communication requests. MPI also defines progress requirements for repeatedly invoked test procedures, making these calls explicit opportunities for the implementation to advance communication operations that have not yet completed [11, Secs. 3.9, 4.7.3–4.7.5].

When the application thread rarely enters MPI during computation and communication does not progress sufficiently on its own, periodic polling can improve communication–computation overlap. A typical approach divides the computation phase into smaller blocks and inserts completion tests between these blocks. If communication has not yet completed, execution returns immediately to computation and the requests are tested again at a later polling point. In this way, communication progress is assisted by periodic MPI test calls made by the application thread rather than relying on an independent asynchronous progress mechanism [14].

The effectiveness of polling depends on the polling frequency. More frequent tests provide additional opportunities for communication progress and may reduce the amount of communication remaining at the final completion stage. However, each polling operation also consumes execution time and interrupts continuous computation. Excessively frequent polling can therefore introduce additional runtime overhead, whereas polling too infrequently may provide insufficient opportunities for communication progress. The polling frequency consequently represents a trade-off between communication advancement and the overhead introduced by the polling operations [21, Sec. 2.4].

In this work, polling-based progress is evaluated separately from asynchronous progress. Nonblocking completion tests are periodically inserted into the computation interval to provide explicit progress opportunities, while computation continues between polling points. This makes it possible to distinguish application-assisted communication progress from asynchronous progress provided independently by the MPI implementation.

2.3. MPI Implementations and Communication Stacks

An MPI program uses the standardized MPI point-to-point interface regardless of the underlying implementation [11]. Internally, however, communication passes through implementation-specific framework layers before reaching shared memory or a network transport [5, 1, 16]. These layers select communication paths according to factors such as peer locality, available devices, and runtime configuration.

In the implementations considered in this work, intra-node communication generally uses a shared-memory path, whereas inter-node communication is routed through a network module and ultimately a transport such as TCP or verbs [5, 1, 16]. Consequently, comparisons between MPI implementations may also reflect differences in internal communication frameworks and selected transport paths unless these factors are controlled explicitly.

Table 2.1 summarizes the principal layers relevant to this study. It shows architec-

tural alternatives rather than implying that all paths are active in one execution. The exact versions and selected paths used in the experiments are given in chapter 3.

Table 2.1.: Conceptual communication-stack layers of the evaluated MPI implementations, based on the respective implementation documentation [7, 1, 16].

Implementation	Framework layer	Intra-node path	Inter-node path
Intel MPI	fabric selection	shared memory	OFI
Open MPI	PML / BTL	self / vader BTL	TCP BTL or UCX/MTL
MPICH	CH4 device	SHM module	OFI or UCX netmod

2.3.1. Intel MPI

Intel MPI separates local and remote communication through its fabric selection. The shared-memory fabric handles ranks on the same node. Inter-node communication enters the Open Fabrics Interfaces (OFI) layer and then libfabric, whose selected provider determines the concrete transport [5, 7]. A TCP provider uses the operating system's TCP and IP network stack, whereas a verbs provider accesses InfiniBand or RoCE through the verbs interface. Provider selection therefore changes the network path without changing the benchmark's MPI calls [7].

2.3.2. Open MPI

Open MPI uses a Point-to-Point Messaging Layer (PML) to coordinate MPI point-to-point requests. With the `ob1` PML, data movement is delegated to Byte Transfer Layers (BTLs): `self` handles communication with the same process, `vader` handles shared-memory communication between local processes, and `tcp` handles remote peers over TCP/IP [1, 17]. Other Open MPI configurations may instead select the `ucx` PML, or the `cm` PML with a Matching Transport Layer (MTL), but these form different communication paths and must not be conflated with `ob1`/TCP measurements [1]. PML and BTL selection also does not by itself imply the presence of a dedicated asynchronous progress thread [18].

2.3.3. MPICH

MPICH's CH4 device separates communication into a shared-memory module (SHM) and a network module (netmod). Local peers can use the SHM path, while remote communication is routed through a netmod such as OFI or UCX [16]. With the OFI

netmod, libfabric provider selection again determines whether the underlying internode path uses TCP, verbs, or another supported fabric. Thus, CH4:OFI identifies the MPI-to-libfabric architecture but does not uniquely identify the physical network transport; the provider must also be reported [16, 7].

2.4. Related Work

Benchmarking communication–computation overlap requires more than measuring communication latency or bandwidth in isolation. Denis and Trahay [3] developed a benchmark suite for nonblocking point-to-point communication and compared overlap across MPI libraries and hardware platforms. Their use of different computation scenarios and low-level traces illustrates how implementation behavior affects the overlap observed by an application. This provides a methodological basis for examining overlap through controlled benchmark experiments.

Research on progress mechanisms provides further context for interpreting benchmark results. Schafer et al. [20] evaluated strong progress in ExaMPI, including its ability to overlap communication with computation without additional application MPI calls. Ruhela et al. [19] developed a thread-based asynchronous progress design that avoids dedicated processor resources and reduces interference from thread scheduling. These studies investigate progress-engine designs; the present thesis instead evaluates the progress configurations available in the installed MPI implementations on the test system.

Zhou et al. [21] analyzed the ambiguity of MPI progress and pointed out that nonblocking operations do not by themselves guarantee continuous progress between initiation and completion. They discussed two common approaches for providing progress during computation: explicit polling with `MPI_Test` and background progress threads, including the overheads associated with both approaches. They further proposed interfaces for explicitly managing MPI progress.

At the application level, Mittermair et al. [13] studied automatic distributed parallelization of structured-grid stencil kernels for ocean-modeling applications. Their approach derives the required halo communication by analyzing data dependencies and automatically inserts the corresponding communication operations into the generated distributed code. Their work shows, at the application level, how halo communication requirements can be derived from the data dependencies of stencil computations, providing relevant background for the present thesis. This thesis focuses on a different question at the runtime level: when a computation window exists between the initiation and completion of nonblocking communication, how outstanding communication progresses under different MPI configurations and how much communication–

computation overlap can ultimately be observed.

The focus of this thesis is a benchmark-based evaluation on the LRZ CoolMUC-4 system. Using an extended Comb benchmark, it investigates how MPI implementation, transport configuration, peer payload, and progress settings affect measured overlap in this environment. The experiments also examine completion methods, computation-window duration, and explicit polling during computation. The contribution is an empirical assessment of these choices under the tested configurations, rather than a new MPI progress algorithm. Its findings characterize this benchmark and system; application-level speedups for a complete ocean model require separate validation.

3. Methodology and Experimental Setup

3.1. Benchmarks and Code Modifications

3.1.1. Comb

Comb is used as the primary benchmark for the experiments in this work. Within each benchmark cycle, nonblocking receives and sends are initiated before the corresponding completion operations are performed. This separation between communication initiation and completion provides the interval in which the study-specific computation and polling mechanisms are inserted. The complete benchmark cycle is described in Section 3.4.3.

The benchmark provides both an MPI backend and a mock backend. The mock path retains the message layout and local packing and unpacking work but performs no MPI transfer. It is therefore used as a control for framework and memory-handling costs. The modifications introduced for this study, including the injected computation phase and polling mechanism, are described below.

3.1.2. Injected Computation

To study communication–computation overlap under controlled conditions, a synthetic compute kernel was added to each communication cycle of Comb. The method draws on the use of dummy computation in the nonblocking-collective overlap tests of OSU Micro-Benchmarks [21]. An adjustable-duration serial computation is inserted after initiating nonblocking communication and before waiting for completion, representing independent application work.

The kernel consists of a sequence of dependent floating-point multiply-add operations. Each block performs 4096 iterations and stores its result in a volatile variable to prevent the computation from being optimized away. The elapsed time of each block is measured with `MPI_Wtime` and accumulated until the target computation duration is reached. This time-driven approach provides an approximately controllable compute interval without requiring a processor-specific iteration count.

Listing 3.1 shows the core implementation, including the surrounding phase timers. The kernel contains no MPI completion or progress calls; `MPI_Wtime` is used only to measure elapsed time.

Listing 3.1: Time-controlled synthetic computation kernel used in Comb.

```
double compute_elapsed_seconds = 0.0;
volatile DataT compute_sink = 1.0;

while (compute_elapsed_seconds < compute_target_seconds) {
    const double start = MPI_Wtime();

    DataT value = compute_sink;
    for (int i = 0; i < 4096; ++i) {
        value = value * 1.0000001192092895507812 5
                    + 0.0000001192092895507812 5;
    }

    compute_sink = value;
    compute_elapsed_seconds += MPI_Wtime() - start;
}
```

The configured value is a lower target rather than an exact execution time. The termination condition is evaluated only after a complete block of 4096 iterations, so the measured duration may exceed the target. It also includes loop-control and timing overhead. The kernel is serial and does not use OpenMP; it is not intended to reproduce the numerical characteristics of a particular ocean-model operator. Instead, it provides a reproducible, adjustable interval in which the progress of outstanding communication can be observed.

3.1.3. Placement of the Injected Computation

The injected computation is placed between the initiation and completion phases of nonblocking communication. Each cycle first posts the nonblocking receives. Comb then allocates and packs the send buffers and posts the nonblocking sends. After all sends have been initiated, the synthetic kernel is executed. The program subsequently completes and unpacks the receives before completing the sends. Figure 3.1 shows the complete cycle, with the injected computation highlighted in green between send posting and receive completion.

This placement creates an explicit potential overlap interval: communication requests have been initiated, but the application thread has not yet entered a request completion operation. The kernel invokes no explicit progress or completion routine such as `MPI_Test` or `MPI_Wait`; it uses `MPI_Wtime` only for timing. A reduction in the subsequent receive completion time is consistent with communication progressing during the compute interval, although phase timings alone cannot establish this directly. If no such progress occurs, most of the remaining communication cost is deferred to the completion phase. The measured timing components and overlap metrics

are defined in section 3.5.

3.2. Hardware and Software Environment

3.2.1. CoolMUC-4

The experiments were conducted on the CoolMUC-4 system of the LRZ Linux Cluster under Slurm. The CoolMUC-4 compute nodes used in this study belong to the Intel Sapphire Rapids node class. The corresponding hardware configuration and interconnect are described in the LRZ CoolMUC-4 system documentation [10]. In the primary experimental configuration, each MPI rank uses a runtime CPU budget of 24 CPU resources.

The primary inter-node experiments use the interactive CoolMUC-4 partition `cm4_inter`. The main configuration uses two compute nodes with one MPI rank per node, resulting in a two-node, two-rank (`2n2r`) setup [9].

3.2.2. Compilers

The Intel MPI and Open MPI configurations use modules from release 24.4.0 of the CoolMUC-4 software stack. For both configurations, Comb was compiled with the Intel oneAPI C++ compiler (`icpx`), version 2024.1.0, using C++17. The selected Open MPI module is the Intel-compiler build. For the MPICH configuration, Comb was compiled with GCC 7.5.0 using C++14 and linked against the locally installed MPICH 4.2.2 library.

All three builds used the CMake `Release` configuration with `-O3 -DNDEBUG` and OpenMP enabled. Separate executables were linked against each MPI implementation. The Intel MPI executable was shared by the verbs and TCP configurations, with the transport selected at runtime.

3.2.3. MPI Implementations

Three MPI implementations are evaluated. Intel MPI is additionally executed with two OFI providers, so the Intel verbs and Intel TCP results represent two transport configurations of the same MPI implementation rather than a fourth implementation. Table 3.1 summarizes the archived software configurations. The transport-selection parameters and asynchronous-progress settings are detailed in section 3.3.

Table 3.1.: MPI implementations and communication configurations used in the experiments.

Implementation	Version or build	Evaluated inter-node path
Intel MPI (verbs)	2021.16.0	OFI verbs
Intel MPI (TCP)	2021.16.0	OFI TCP
Open MPI (TCP)	4.1.8-intel24	`ob1` with TCP BTL
MPICH (TCP)	4.2.2, local GCC build	CH4:OFI with TCP provider

3.2.4. Build Configuration

All configurations use the same modified Comb source and enable the MPI and OpenMP execution backends. The synthetic computation is controlled at runtime, so the same executable is used for the paired runs with and without injected computation. Newly generated runs pass a common communication cutoff of 250 explicitly. The build scripts and accompanying documentation in the project code repository specify the compiler selections, CMake options, and executable paths needed to reproduce the builds.

3.3. MPI Runtime Configuration

3.3.1. Communication Stacks and Providers

Intel MPI uses `I_MPI_FABRICS=shm:ofi`, with `I_MPI_OFI_PROVIDER=verbs` or `I_MPI_OFI_PROVIDER=tcp` selecting the provider for the two transport configurations.

Open MPI uses `OMPI_MCA_pml=ob1` and `OMPI_MCA_btl=self,vader,tcp`. The locally built MPICH 4.2.2 uses the CH4:OFI device with `FI_PROVIDER=tcp`.

3.3.2. Verbs and TCP

The Intel MPI verbs and TCP configurations keep the MPI implementation and benchmark configuration fixed while changing only the OFI provider, allowing the effect of provider selection to be examined separately.

Open MPI and the MPICH build used in this study both rely on TCP for inter-node communication. The MPICH results are therefore primarily compared with the other TCP-based paths.

For the locally built MPICH configuration, a verbs provider was also tested. After the initial library configuration issue had been resolved, OFI initialization on the

mlx5_0 device still failed with errors such as Failed to modify UD QP to INIT ... Operation not permitted, eventually causing MPI_Init_thread to abort. This indicates that the automatically selected RDMA path required permissions or resources that were not available in the current job environment. The subsequent MPICH run scripts therefore use FI_PROVIDER=tcp by default, while still allowing the provider to be overridden at runtime. Consequently, no MPICH verbs measurements are included in this study.

3.3.3. Asynchronous Progress Configuration

For Intel MPI, asynchronous progress is controlled with I_MPI_ASYNC_PROGRESS; the number and affinity of progress threads are set through I_MPI_ASYNC_PROGRESS_THREADS and I_MPI_ASYNC_PROGRESS_PIN. Both Intel transports use the same thread-count and binding logic. Communication uses the first progress thread by default; distributing work across progress threads requires explicit communicator-to-thread assignment through a communicator info key, whose default name is thread_id [6, 4]. The evaluated progress-enabled Intel MPI configurations request one progress thread per rank and perform no communicator-to-thread assignment.

Open MPI 4.1.8 uses OMPI_MCA_btl_tcp_progress_thread=1 in the enabled cases. In this version, the TCP BTL treats the parameter as an enable switch and starts a single background progress thread; it is not a thread-count parameter [18]. This requests TCP-BTL-specific progress, not a transport-independent progress mode. The scripts also set mpi_yield_when_idle=0, which controls processor yielding while waiting for MPI communication and does not itself enable a progress thread. The script argument named async_threads determines how many CPU IDs are withheld from the application's OpenMP places. It does not configure that many Open MPI progress threads. No explicit binding of the TCP progress thread to those reserved CPU IDs is performed by the launcher script; the reserved list therefore does not establish the thread's actual placement.

In the MPICH 4.2.2 configuration used in this study, asynchronous progress is enabled through MPIR_CVAR_ASYNC_PROGRESS=1, which creates one background progress thread per rank. This thread can advance outstanding communication while the application performs computation without entering MPI calls. The control variable is Boolean and does not expose the number of background progress threads as a tunable runtime parameter [15]. Its target CPU is set with MPIR_CVAR_PROGRESS_THREAD_AFFINITY. Progress-thread count is therefore not treated as an experimental variable for MPICH. MPICH extensions provide finer-grained application-controlled progress mechanisms [22, 21], but these non-standard APIs are outside the scope of the runtime configuration evaluated here.

3.3.4. CPU and Rank Binding

In the two-node configuration, one MPI rank is placed on each node. The launcher scripts expand the allowed logical-CPU list and select different subsets by position. Selecting different CPU IDs alone does not guarantee that these CPUs correspond to distinct physical cores. The socket and NUMA labels used in this work describe the selected CPU sets rather than providing a runtime measurement of the actual placement of the progress thread.

For Intel MPI, the configurations without asynchronous progress use Slurm CPU-core binding. When asynchronous progress is enabled, Slurm binding is disabled, the application is restricted to the intended CPU set using `taskset`, and a separate list of target CPUs for the progress thread is supplied through `I_MPI_ASYNC_PROGRESS_PIN`.

MPICH retains Slurm core binding and selects a target CPU for the progress thread. At the same time, application-only CPU sets are constructed through `OMP_PLACES`, with `OMP_PROC_BIND=close` used to control application-thread placement.

In the progress-enabled Open MPI configuration, the application threads are restricted to the selected OpenMP CPU places, leaving one CPU unused by the application. However, the TCP progress thread is not explicitly pinned to that reserved CPU. Therefore, the reserved CPU represents an application-side resource reservation rather than a verified placement of the progress thread.

3.3.5. `wait_all` and `test_all` Completion Variants

The receive and send completion methods are varied together: a `wait_all` case uses `MPI_Waitall`, while a `test_all` case repeatedly calls `MPI_Testall` until all requests have completed. Both completion methods execute after the injected computation. The test loop therefore belongs to the final completion phase and does not introduce polling during computation. Receive and send posting remain unchanged. Table 3.2 lists the four cases evaluated in this thesis, distinguishing configured progress-thread counts from reserved CPU IDs. These quantities are not interchangeable across MPI implementations.

For Intel MPI, the progress-enabled configurations use 23 application CPU IDs and one CPU ID in the progress pin list. MPICH likewise reserves one CPU ID for progress, leaving 23 application places. Open MPI excludes one CPU ID from the application's OpenMP places without explicitly binding its TCP progress thread to that CPU. The application-thread counts in the table are runtime settings; the injected computation itself is serial.

The thesis uses consecutive Case IDs 01–04. They correspond, respectively, to the archived experiment IDs 01, 03, 06, and 09; raw data and figure filenames retain these

Table 3.2.: Per-rank completion and CPU-resource configurations. Intel columns apply to both verbs and TCP. Open MPI reports reserved CPU IDs, not a configured progress-thread count.

Case	Completion	App. threads	Intel progress	MPICH progress	Open MPI reserved CPUs
01	`wait_all`	24	off	off	0 (off)
02	`test_all`	24	off	off	0 (off)
03	`wait_all`	23	1	1	1 (on)
04	`test_all`	23	1	1	1 (on)

archived identifiers for traceability.

3.4. Workloads and Experiment Matrix

3.4.1. Grid, Ghost Width, and Variables

The message-size sweep uses two fixed three-dimensional global grids and varies the number of grid variables to increase the communication volume. All configurations use a ghost width of one cell in each spatial direction (`ghost=1_1_1`) and periodic boundary conditions in all three directions (`periodic=1_1_1`).

The reference configuration uses two nodes with one MPI rank per node and a `divide=2_1_1` domain decomposition. The global domain is divided only in the x direction. The first sweep uses a global grid of `grid=8_8_8` to cover the small-message range. Each rank owns an interior subdomain of $4 \times 8 \times 8$ cells, and the reference variable counts are

$$v_{8^3}^{2n2r} \in \{1, 2, 3, 4, 6, 8, 10, 12\}. \tag{3.1}$$

The second sweep uses `grid=32_32_32` to cover medium and large messages. Each rank owns an interior subdomain of $16 \times 32 \times 32$ cells, and the reference variable counts are

$$v_{32^3}^{2n2r} \in \{1, 2, 4, 8, 16, 32, 64, 128, 192, 256, 336\}. \tag{3.2}$$

3.4.2. Message Sizes

Comb aggregates all halo data addressed to the same partner into one send buffer. The payloads reported here therefore denote the complete aggregated payload of an

inter-node `MPI_Isend`, including face, edge, and corner data, rather than the payload of a single halo face. For the two-rank reference topology, both periodic x-direction boundaries address the other rank. With a cubic global grid of side length N, one ghost layer, double-precision values, and v variables, the aggregated inter-node partner payload is

$$P_{\text{peer}}(N, v) = 2(N+2)^2 \times 8v \text{ bytes.} \tag{3.3}$$

For the 8^3 grid, each additional variable increases the payload by 1,600 bytes. For the 32^3 grid, one variable produces 18,496 bytes of aggregated payload. Together, the two grid configurations form the continuous reference sweep from 1.56 KiB to 5.93 MiB shown in Table 3.3.

3.4.3. Cycles and Compute Targets

Every point in the message-size sweep executes 100 timed communication cycles and is repeated in ten independent runs. Each run contains a matched pair of executions. The `no_midcomm` execution sets `compute_target_us=0` and provides the no-compute communication reference. The `midcomm` execution uses the same communication and placement settings but inserts the synthetic computation between communication initiation and completion.

The Comb option `-cycles` specifies the number of timed halo-exchange iterations executed by each rank for every tested communication and execution policy. One cycle traverses the complete benchmark path: `pre-comm` initializes the local interior cells; `post-recv` allocates receive buffers and posts the nonblocking receives; `post-send` allocates and packs send buffers and posts the nonblocking sends; `mid-comm` executes the injected computation when its target is positive; `wait-recv` completes receives and performs the associated unpacking and buffer handling; `wait-send` completes the sends; and `post-comm` restores the mesh state for the next iteration. Communication-object construction and allocation occur before this timed cycle loop. Comb also executes $\max(1, \lfloor N_c/10 \rfloor)$ correctness cycles before clearing the phase timer; these exercise the same exchange path but are excluded from the reported phase statistics. Consequently, a summary count of PN_c for a phase denotes N_c samples from each of P ranks, not PN_c independent experimental repetitions. Independent runs, rather than cycles, are therefore used for statistical aggregation.

Unless otherwise specified in the following sections, the target duration of the injected computation is set approximately equal to the corresponding reference communication time T_{comm}. This reference is selected separately for each MPI implementation, transport configuration, and peer payload, rather than applying one fixed duration to the complete sweep. All overlap metrics use the measured `mid-comm` duration, not the

Table 3.3.: Aggregated inter-node partner payloads for the complete message-size sweep in the two-rank reference topology.

Global grid	Ghost width	Variables	Peer payload
$8 \times 8 \times 8$	$1 \times 1 \times 1$	1	1.56 KiB
$8 \times 8 \times 8$	$1 \times 1 \times 1$	2	3.13 KiB
$8 \times 8 \times 8$	$1 \times 1 \times 1$	3	4.69 KiB
$8 \times 8 \times 8$	$1 \times 1 \times 1$	4	6.25 KiB
$8 \times 8 \times 8$	$1 \times 1 \times 1$	6	9.38 KiB
$8 \times 8 \times 8$	$1 \times 1 \times 1$	8	12.50 KiB
$8 \times 8 \times 8$	$1 \times 1 \times 1$	10	15.63 KiB
$8 \times 8 \times 8$	$1 \times 1 \times 1$	12	18.75 KiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	1	18.06 KiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	2	36.13 KiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	4	72.25 KiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	8	144.50 KiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	16	289.00 KiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	32	578.00 KiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	64	1.13 MiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	128	2.26 MiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	192	3.39 MiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	256	4.52 MiB
$32 \times 32 \times 32$	$1 \times 1 \times 1$	336	5.93 MiB

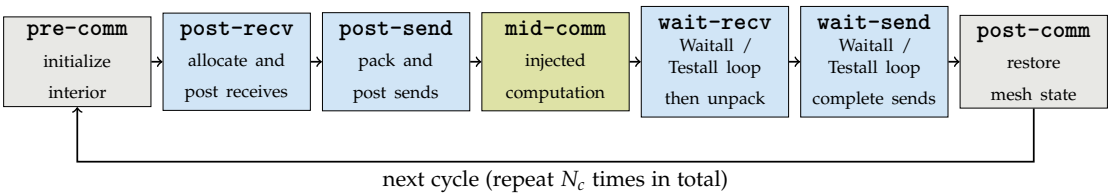


Figure 3.1.: Phases executed by each rank in one timed Comb communication cycle. Gray boxes denote local mesh operations, blue boxes MPI communication phases, and the green box the injected computation window. Receive and send completion use either Waitall or a Testall loop; the timer names remain `wait-recv` and `wait-send` in both cases. The return arrow connects successive cycles.

requested target, because the block-based compute loop can overshoot its minimum target.

3.4.4. Node Count and Process Topology

The reference topology, `2n2r_c24`, uses two nodes and one MPI rank per node. Its `divide=2_1_1` decomposition partitions the global grid only along x. Under periodic boundary conditions, the x exchange is inter-node communication, whereas the undecomposed y and z directions produce MPI self-messages.

3.5. Timing and Evaluation Metrics

3.5.1. Timing Components and Baselines

Comb records one timer sample per rank and cycle for the phases `pre-comm`, `post-recv`, `post-send`, `mid-comm`, `wait-recv`, `wait-send`, and `post-comm`. The `mid-comm` timer measures the injected compute interval. The `wait-recv` phase contains receive completion followed by unpacking and buffer release; it therefore measures exposed receive-side work and is not a pure network timer. The `bench-comm` timer encloses all timed cycles and yields one value per rank. Its maximum across ranks represents the benchmark critical path.

Let $\bar{W}_0$ and $\bar{W}_c$ be the mean `wait-recv` times per call without and with injected computation, respectively, and let $\bar{C}$ be the mean measured `mid-comm` time per call. Means are taken over the same rank–cycle sample set within one run. Let B_0 and B_c denote the maximum `bench-comm` values across ranks for the corresponding pair.

3.5.2. Mock-Corrected Communication Cost

To separate MPI-specific cost from Comb's local framework and memory-handling costs, an additional matched baseline measurement is performed without injected computation. For each workload, the MPI and mock backends are enabled separately and run as independent program executions. MPI-only and Mock-only runs with matching identifiers use the same allocation, grid, message layout, execution policy, cycle count, and completion method. The mock backend follows the corresponding benchmark path but performs no MPI transfer. Let $\bar{W}_0^{\mathrm{MPI}}$ and $\bar{W}_0^{\mathrm{mock}}$ denote their mean `wait-recv` times. The Mock-corrected communication cost is defined as

$$\widehat{T}_{\mathrm{comm}} = \bar{W}_0^{\mathrm{MPI}} - \bar{W}_0^{\mathrm{mock}}. \tag{3.4}$$

This quantity estimates the incremental exposed receive-side cost associated with enabling MPI communication after subtracting the local framework and memory-handling cost represented by the mock backend. It is therefore used as the communication cost reference for the subsequent message-size and overlap analyses, but it is not interpreted as pure network-transfer latency. The measured values of $\widehat{T}_{\mathrm{comm}}$ for each message size and MPI/transport configuration are reported in Table 4.2.

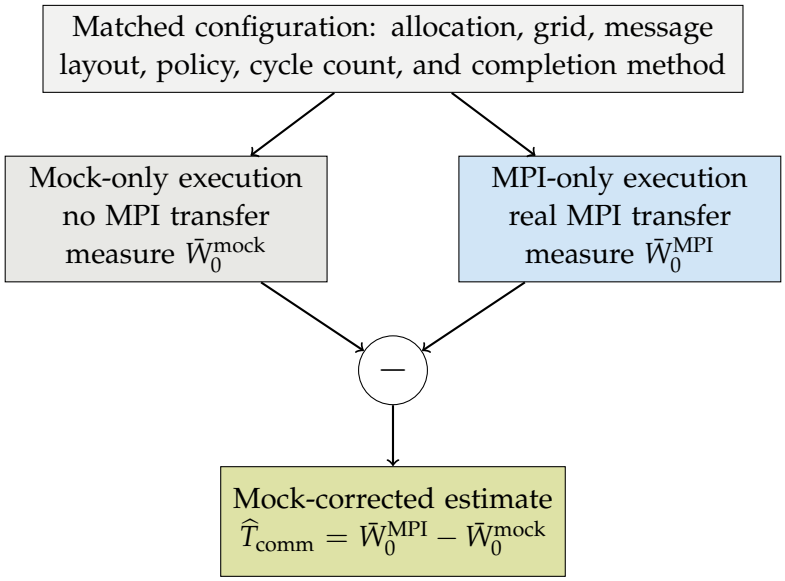


Figure 3.2.: Matched, independent MPI-only and Mock-only executions used to estimate the exposed MPI-specific communication cost. The result is not interpreted as pure network-transfer latency.

This subtraction removes the local work represented by the mock path and retains

the incremental cost observed when MPI communication is enabled. It is therefore an estimate of MPI-specific exposed communication cost, not a direct measurement of pure network-transfer latency. In particular, MPI may also change synchronization, protocol processing, cache state, and memory behavior. Separate program executions avoid directly carrying one backend's in-process allocator and cache state into the other. Temporal drift between the paired executions can nevertheless remain; repeated run-level differences are therefore reported rather than subtracting values pooled across unrelated runs.

3.5.3. Operational Measure of Overlap

The observed receive-wait reduction between matched runs is

$$\Delta W = \bar{W}_0 - \bar{W}_c. \tag{3.5}$$

To assess communication–computation overlap using application-visible timings, this thesis defines the normalized receive-wait reduction as

$$R_{\text{wait}} = \frac{\bar{W}_0 - \bar{W}_c}{\bar{C}} \times 100\%, \qquad \bar{C} > 0. \tag{3.6}$$

This metric expresses the reduction in receive-side waiting relative to the measured computation time. Throughout the following analyses, measured overlap refers to this operational metric R_{wait}, rather than the fraction of communication time hidden by computation.

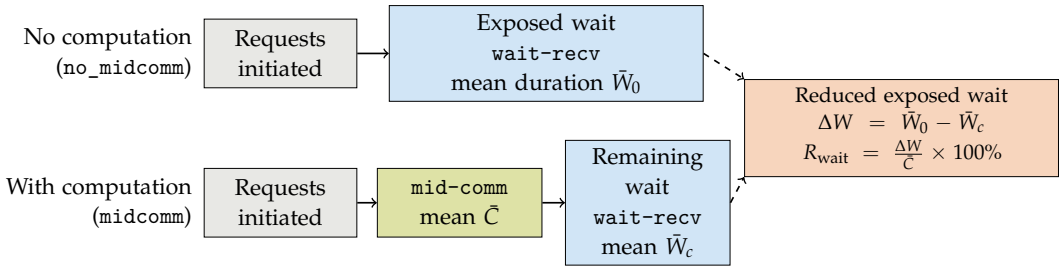


Figure 3.3.: Schematic comparison used for the normalized receive-wait reduction. The upper and lower rows are matched, separate runs; durations are not drawn to scale.

A positive value means that less receive-side waiting remains after the compute interval. A negative value means that the measured wait increased. A value of 100% means that the waiting-time reduction equals the measured computation time; it does not necessarily mean that all communication has been hidden. Since the measured

phase also includes unpacking and buffer release, the metric may reflect changes in local processing and runtime conditions in addition to communication progress. Values above 100% are possible because inserting computation can alter protocol timing, unpacking cost, scheduling, and contention, while timing variability between matched runs can also affect the difference. Such values must not be interpreted as more than complete physical overlap. This metric is an operational response of the measured phase, not a direct observation of network progress in isolation or a measure of overall application speedup.

3.5.4. Statistical Aggregation and Variability

Phase means are first computed within each run from the rank–cycle samples reported by Comb. Critical-path quantities use the maximum rank-level `bench-comm` time. The formulas above are then evaluated for each matched run, preserving `midcomm` and `no_midcomm` as a pair. The ten resulting run-level values are treated as the independent observations for reporting central tendency and variability. Individual cycles are not treated as independent experimental repetitions. Alongside derived percentages, absolute phase times and the spread among runs are reported so that unstable small-message measurements remain visible.

3.6. Experimental Procedure

3.6.1. Staged Experimental Design

The experimental campaign is organized as a sequence of calibration and measurement stages rather than as one undifferentiated parameter sweep. First, an exploratory message-size scan is performed on the two-node reference topology. The 19 payloads defined in the workload matrix span 1.56 KiB to 5.93 MiB and are used to identify message-size regimes, unstable points, and implementation-specific changes in communication behavior.

Second, communication costs are measured without injected computation. For every MPI implementation and payload, matched MPI-only and Mock-only executions are run independently, and their `wait-recv` difference is used to estimate $\widehat{T}_{\mathrm{comm}}$. This stage also checks whether the subtraction is stable and physically interpretable across independent runs. Pilot measurements that used both backends in one process are retained only as diagnostic evidence for execution-order effects and are not pooled with the formal independent-backend baselines.

Third, the measured communication cost is used to select the injected-computation target for each implementation and payload. The main message-size sweep uses a tar-

get comparable to the exposed communication interval; additional ratio experiments use fractions and multiples of that interval to study the computation–communication trade-off. Finally, matched `no_midcomm` and `midcomm` runs measure overlap under otherwise identical settings.

3.6.2. Warm-up

Before the timed benchmark, Comb initializes the communication objects and executes a correctness phase of $\max(1, \lfloor N_c/10 \rfloor)$ communication cycles. For the main setting of $N_c = 100$, this gives ten untimed cycles. These cycles exercise the same communication path and validate exchanged data. A barrier follows the correctness phase, the phase timer is cleared, and only then does the `bench-comm` region begin. The warm-up is thus excluded from all timing metrics. However, this short correctness phase is intended primarily for data validation and initial path activation; it is not assumed to establish a steady state for memory allocation, page placement, caches, MPI protocols, or the network. Preliminary measurements showed first-run and execution-order effects even after these cycles. Consequently, independent runs, rather than the built-in correctness cycles, are used to assess stability, and visible first-run shifts are retained and reported rather than silently discarded.

3.6.3. Repetitions and Paired Runs

Each workload point is executed ten times for both compute modes. A valid pair has identical grid, variable count, MPI implementation, transport, progress setting, completion method, thread count, and binding; only the compute target differs. Pairing controls configuration drift and allows the reference quantities in section 3.5 to be formed without combining unrelated executions.

3.6.4. Execution Order

Generated job bundles execute their child experiments serially within a single SLURM allocation. This reuses the same allocated nodes for the bundled payloads, compute modes, and repetitions and avoids interpreting allocation changes as an experimental factor. The generated order is retained in the run metadata. The Mock-corrected cost uses separate MPI-only and Mock-only program executions with matched run identifiers. Their launch order and timestamps are retained so that any remaining temporal drift is visible. The ten independent run pairs, rather than individual cycles, form the unit of variability.

3.6.5. Data Filtering

Analysis first identifies the cases present in both the computation and communication-cost directories. A case is excluded from a derived metric if either summary is missing, if the communication-policy block is absent, or if any required timer is unavailable. Non-positive compute times or serial baselines are rejected because the corresponding ratios are undefined. Negative or unexpectedly large normalized values do not, by themselves, establish that a run is invalid; such values remain diagnostically relevant.

Substantial run-to-run timing variation was observed during the experiments. Network contention and changes in shared-cluster load or scheduling are possible contributors, but the timing summaries alone do not establish their individual effects. During run selection, communication-reference timings are compared with the reference for the same workload and configuration. Runs or batches that depart markedly from that reference are flagged for inspection and may be excluded from the selected dataset or remeasured. This is a manual review rather than an automatic rejection rule with a predefined numerical threshold. Differences in cycle count, execution mode, and runtime configuration must also be considered before attributing a shift to external interference.

3.7. Reproducibility and Threats to Validity

Reproducibility is supported by generated job scripts, explicit runtime settings, recorded benchmark parameters, CPU-binding configurations, and retained per-run summaries. The software versions, transport configuration, topology, payload, cycle count, and compute target are recorded for each experiment. The accompanying results repository is linked in section A.1.

Several limitations must be considered when interpreting the results. First, the injected computation is synthetic and serial and therefore does not reproduce the instruction mix or memory behavior of a complete application. Second, `wait-recv` includes completion and local unpacking work and must not be interpreted as pure network-transfer time. Third, Intel MPI is evaluated with both verbs and TCP, whereas Open MPI and MPICH use TCP, so transport effects cannot always be separated completely from implementation effects. Fourth, asynchronous progress consumes CPU resources and may compete with application execution. Finally, the experiments are executed on a shared cluster, and each data point is repeated ten times independently. These repetitions provide a more reliable estimate of run-to-run variability, although microsecond-scale measurements can still remain sensitive to system noise. Conclusions are therefore based on repeated trends, absolute timings, and Mock controls rather than isolated percentage peaks.

4. Experimental Results and Analysis

This chapter presents the results by experimental factor. It first validates the measurement procedure and then isolates the effects of message size, MPI implementation, transport, progress mechanism, CPU-resource configuration, and compute-window duration. Unless stated otherwise, each comparison changes only the factor named in the corresponding section.

4.1. Measurement Validation

Each reported result is based on ten independent experimental runs. Repeated measurements capture run-to-run variability, but do not eliminate random timing fluctuations. Variations in network conditions, operating-system scheduling, and other runtime effects may produce occasional peaks in the measured communication times and the resulting overlap values. The measurements do not isolate the contribution of each of these possible sources.

All measured values, including such peaks, are retained in the reported statistics; observations are not discarded solely because they deviate markedly from the other runs. The mean and the run-to-run range therefore preserve the random variability observed across the ten runs. The subsequent analysis considers this variability when interpreting differences between configurations and trends across message sizes.

Table 4.1 illustrates this variability for Intel MPI TCP at a peer payload of 1.56 KiB. Although each case contains ten independent runs, substantial differences remain between individual measurements.

Case 01 ranges from 21.12% to 204.34%, with a mean of 57.23%. Run 03 produces a pronounced peak of 204.34%, while the other nine measurements lie between 21.12% and 61.69%. Case 03 also exhibits substantial variation, ranging from 36.54% to 133.35%, with a mean of 71.65%; runs 01 and 06 both exceed 100%. These results demonstrate that ten independent runs do not eliminate large timing fluctuations under the measured runtime conditions. Values above 100% under this normalization by the measured computation time do not imply physical overlap beyond the compute window. All ten observations, including the peaks, are retained in the statistics so that the reported results reflect this random variability rather than suppressing it through outlier removal.

4.2. Message Size and MPI Implementation

4.2.1. Mock-Corrected Communication Cost

Table 4.2 reports the Mock-corrected communication cost $\widehat{T}_{\text{comm}}$, defined in section 3.5, for the 19 payloads of the two-node, two-rank `cm4_inter` reference configuration. The results correspond to Case 03 with asynchronous progress enabled, 23 application threads, `wait_all` completion, and the fully sequential `seq/seq-seq` execution policy. Payload denotes the complete aggregated message sent to the inter-node partner.

Across the complete sweep, the exposed communication cost generally increases with peer payload, but the magnitude and rate of increase differ substantially between the MPI implementations and transport configurations. Intel MPI verbs has the lowest cost over most of the small-message region, while Open MPI TCP shows markedly higher communication cost at the smallest payloads. As the payload increases, the differences between the TCP-based configurations become more pronounced, with Intel MPI TCP showing the highest communication cost at the upper end of the sweep. The measured Mock-corrected communication cost does not increase linearly with message size and can even decrease over some payload intervals, as observed for Open MPI TCP.

These differences in communication baseline are important for the subsequent overlap comparison because, even at the same peer payload, the communication interval available to be hidden by computation differs between MPI implementations. The implementation- and payload-specific compute targets defined in section 3.5 are therefore retained in the subsequent overlap analysis rather than applying a common compute duration across the complete message-size sweep.

The values in Table 4.2 are used primarily to characterize the communication-cost scale and to support compute-window calibration, rather than as high-precision estimates of the population mean.

4.2.2. Overlap across Message Sizes

Open MPI

Beyond 144.50 KiB, the overlap decreases progressively across several consecutive payloads. The mean falls from approximately 82% at 144.50 KiB to about 68% at 289 KiB, 40% at 578 KiB, and approximately 20% at 1.13 MiB. Because this reduction extends across multiple neighboring message sizes, it represents a systematic change in the observed message-size behavior rather than an isolated data point.

For payloads above 1.13 MiB, the mean overlap remains at a substantially lower level and shows only a limited recovery at larger message sizes. At the same time, the

Table 4.1.: Individual overlap results for Intel MPI TCP at a peer payload of 1.56 KiB. All ten runs use the same Mock-corrected communication cost of 30.282 μs as the normalization reference. The final rows report the minimum, mean, and maximum across the ten runs.

Run	Case 01 (%)	Case 03 (%)
run_01	21.12	113.27
run_02	39.16	39.87
run_03	204.34	75.97
run_04	61.69	67.58
run_05	28.57	67.37
run_06	45.53	133.35
run_07	37.72	57.11
run_08	39.91	57.57
run_09	40.63	67.85
run_10	53.59	36.54
Min	21.12	36.54
Mean	57.23	71.65
Max	204.34	133.35

Table 4.2.: Mock-corrected communication cost for Case 03 with the `seq/seq-seq` policy in the `cm4_inter` two-node, two-rank configuration. Values are in μs.

Peer payload	Intel verbs	Intel TCP	MPICH TCP	Open MPI TCP
1.56 KiB	10.322	30.282	26.824	124.698
3.13 KiB	12.789	32.338	32.167	185.412
4.69 KiB	18.389	31.142	30.507	214.819
6.25 KiB	24.944	41.584	28.594	210.408
9.38 KiB	27.701	46.703	36.017	274.870
12.50 KiB	33.171	50.929	37.435	280.118
15.63 KiB	37.382	51.662	37.095	311.566
18.06 KiB	37.322	62.542	50.211	345.424
18.75 KiB	42.687	62.252	30.838	317.220
36.13 KiB	35.889	58.718	44.679	493.553
72.25 KiB	73.893	79.006	76.149	248.083
144.50 KiB	100.000	64.543	177.998	208.448
289.00 KiB	170.000	198.836	178.221	259.094
578.00 KiB	230.676	373.755	269.086	200.423
1.13 MiB	501.483	838.070	405.400	267.775
2.26 MiB	746.096	1383.583	993.808	851.659
3.39 MiB	963.993	2509.543	1493.140	1143.607
4.52 MiB	1167.395	3420.671	2136.178	2061.891
5.93 MiB	2076.622	4090.282	3456.913	2980.504

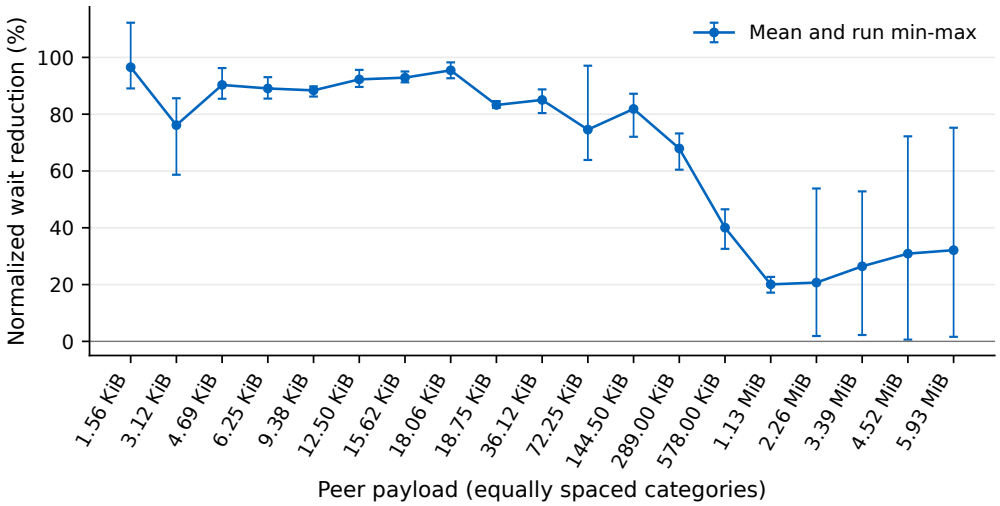


Figure 4.1.: Overlap across the 19 peer payloads for Open MPI in the two-node, two-rank `cm4_inter` configuration for Case 03. Open MPI uses the OB1 PML with the TCP BTL for inter-node communication, with asynchronous TCP progress enabled, 23 application threads, one CPU reserved for the progress thread, `wait_all` completion, and the `seq/seq-seq` execution policy.

run-to-run ranges are generally larger than in the small-message region, indicating reduced stability of the overlap behavior for large payloads.

The Open MPI TCP parameters provide additional context for this behavior. The parameters were queried using `ompi_info` in the same Open MPI environment used for the experiments.[1] The TCP BTL reports an eager limit of 64 KiB and a maximum send size of 128 KiB. The first sampled payload above the eager limit is 72.25 KiB, where a local reduction in overlap is visible. However, the more pronounced and sustained decrease begins only at larger payloads, beyond approximately 144.50 KiB.

This indicates that the eager-to-rendezvous transition alone is not sufficient to explain the complete large-message trend. Although the onset of the large-message regime is consistent with the first local change in overlap, the continued degradation at larger payloads suggests that additional large-message effects, such as sustained progress requirements and increased data movement, may also contribute. The present measurements do not isolate these mechanisms individually, so no single protocol threshold is assigned as the definitive cause of the observed decrease.

MPICH TCP

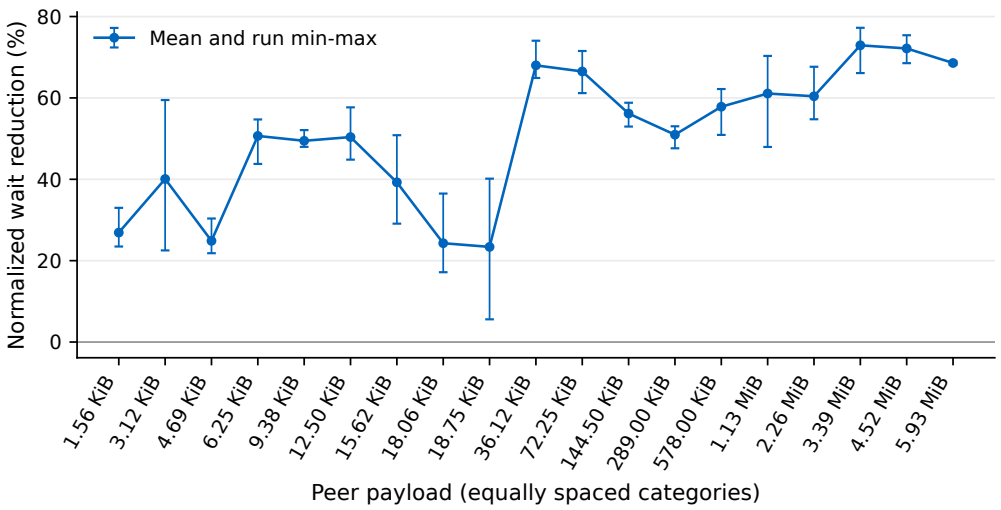


Figure 4.2.: Overlap across the 19 peer payloads for MPICH TCP in the two-node, two-rank `cm4_inter` configuration for Case 03, using `wait_all` completion and the `seq/seq-seq` execution policy.

For MPICH TCP, the most notable message-size dependence appears in the small-

[1] `ompi_info -param btl tcp -level 9`

message region, where two pronounced local turning points are visible. The overlap varies considerably across the smallest payloads rather than following a monotonic trend.

Beyond the small-message region, MPICH achieves both higher and more stable overlap. From approximately 36.12 KiB onward, the mean overlap generally remains at a moderate to high level, and the large-message region shows substantially less variation in the overall trend. In the MiB range, the mean overlap is mostly maintained around 60%–70%, indicating that MPICH preserves comparatively strong overlap even as the payload increases.

The MPICH results therefore differ from the Open MPI behavior observed above: the main irregularities are concentrated at small payloads, whereas larger messages exhibit a more stable and consistently higher degree of communication–computation overlap.

Intel MPI: verbs and TCP

Figures 4.3 and 4.4 show the message-size dependence of measured overlap for Intel MPI verbs and TCP with asynchronous progress enabled (Case 03). Both configurations exhibit irregular overlap at small payloads and generally higher overlap at larger payloads, although their local variations differ.

For Intel MPI verbs, the mean overlap decreases from approximately 72% at 3.12 KiB to 57% at 4.69 KiB and 46% at 6.25 KiB, before recovering at the following payloads. A further region of lower overlap extends from 18.06 KiB to 72.25 KiB. The mean is approximately 59% at 36.12 KiB and 41% at 72.25 KiB, followed by a pronounced increase to approximately 95% at 144.50 KiB.

For Intel MPI TCP, local reductions are visible at 4.69 KiB and around 18.06–18.75 KiB. The mean overlap increases from approximately 59% at 18.75 KiB to 89% at 36.12 KiB, remains around 85% at 72.25 KiB, and reaches approximately 98% at 144.50 KiB. Thus, the recovery occurs at a smaller payload than for verbs.

From 144.50 KiB onward, both transports maintain comparatively high measured overlap across the remaining sweep. Intel MPI verbs generally remains near 100%, while TCP mostly lies between approximately 83% and 103%. Neither transport shows a sustained decline toward zero in the large-payload region under the tested progress-enabled configuration.

4.2.3. Cross-Implementation Comparison

The message-size sweeps reveal two qualitatively different overlap patterns across the evaluated MPI implementations. Intel MPI and MPICH both exhibit pronounced ir-

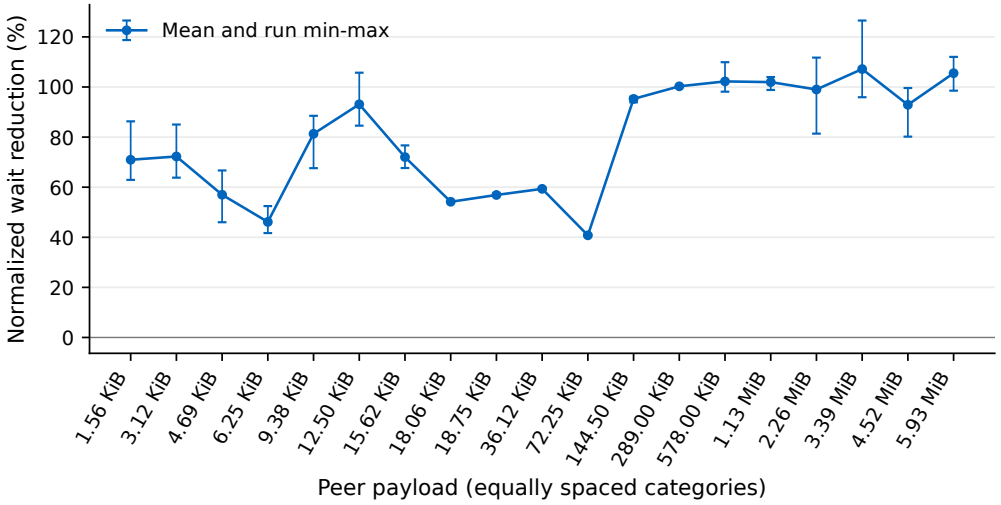


Figure 4.3.: Overlap across the 19 peer payloads for Intel MPI using the verbs transport in the two-node, two-rank `cm4_inter` configuration for Case 03.

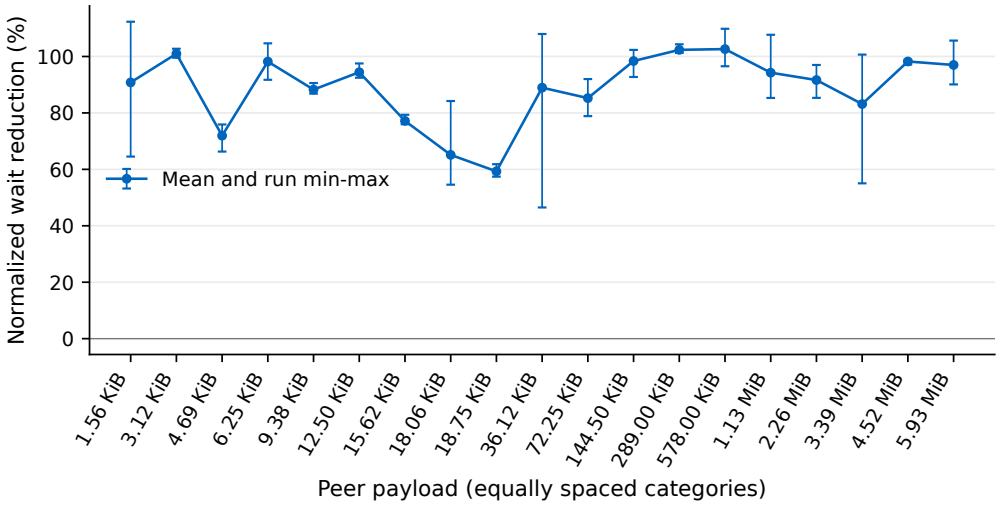


Figure 4.4.: Overlap across the 19 peer payloads for Intel MPI using TCP transport in the two-node, two-rank `cm4_inter` configuration for Case 03.

regularities in the small-message region, while medium and large payloads generally achieve higher and more stable overlap. For Intel MPI, this overall pattern is observed with both verbs and TCP transport, indicating that the broad message-size dependence is not specific to a single transport path.

Open MPI shows a different large-message behavior. Its overlap remains high for small and medium payloads but decreases progressively once the payload enters the larger-message range. Consequently, the relative behavior of the implementations changes with message size: Open MPI performs strongly in the small-message region, whereas Intel MPI and MPICH maintain substantially higher overlap for large payloads.

4.2.4. Asynchronous Progress

Open MPI

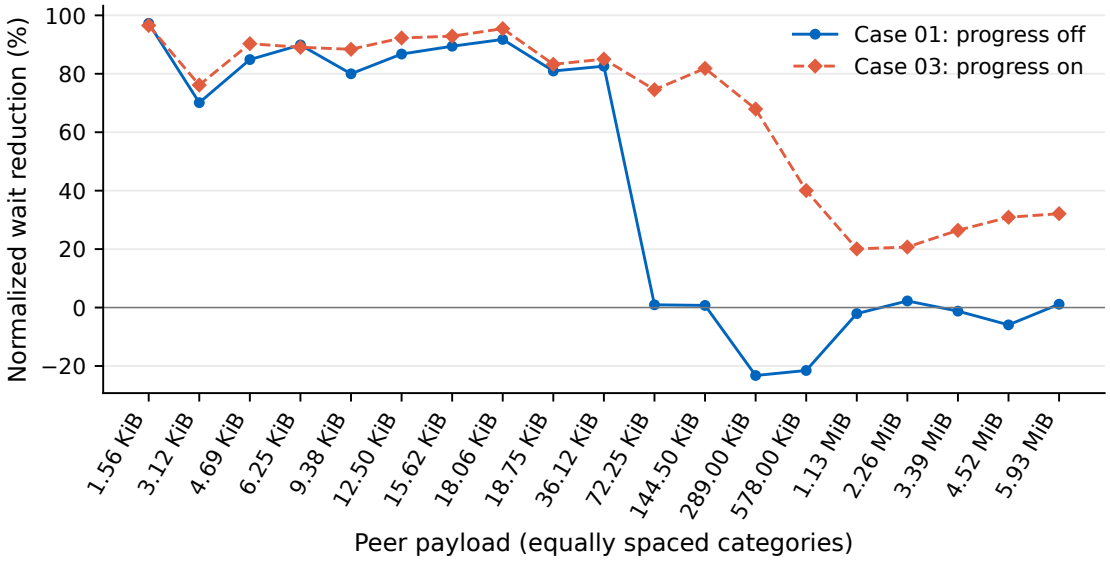


Figure 4.5.: Open MPI TCP: measured overlap across 19 peer payloads with asynchronous progress disabled (Case 01, solid line with circles, 24 application threads) and enabled (Case 03, dashed line with diamonds, 23 application threads and one CPU reserved for progress). Both cases use `wait_all` and `seq/seq-seq` in the two-node, two-rank `cm4_inter` configuration.

For Open MPI, asynchronous progress is not controlled by a single global on/off switch. In the present experiments, Case 03 represents the progress-thread configu-

ration by enabling a dedicated TCP BTL progress thread and reserving one CPU for it.[2]

As shown in Figure 4.5, Case 01 and Case 03 exhibit similar behavior in the small-message region. Up to approximately 36.12 KiB, both configurations generally maintain high overlap, with most values between about 70% and 95%. This indicates that the TCP BTL progress-thread configuration has a limited effect on Open MPI overlap for smaller message payloads.

The first clear divergence occurs at 72.25 KiB. In Case 01, without a dedicated TCP BTL progress thread, the mean overlap drops sharply from approximately 83% at 36.12 KiB to nearly zero at 72.25 KiB and remains close to zero for most subsequent larger payloads. In contrast, Case 03 retains substantial overlap over the same message-size range. Although the Case 03 overlap decreases as the payload increases, it remains consistently higher than that of Case 01 throughout the medium- and large-message region.

These results indicate that the Open MPI TCP BTL progress-thread configuration has a limited effect in the small-message region, but substantially improves communication–computation overlap for medium and large message payloads.

MPICH

For MPICH, asynchronous progress is controlled through `MPIR_CVAR_ASYNC_PROGRESS`. Case 01 disables asynchronous communication progress, whereas Case 03 enables the MPICH asynchronous progress thread.[3]

As shown in Figure 4.6, both MPICH configurations exhibit relatively irregular overlap behavior in the small-payload region. Below 36.12 KiB, the effect of asynchronous progress is not consistent: overlap varies substantially with peer payload size, and neither configuration consistently outperforms the other across the entire small-payload range.

A clearer difference between the two configurations emerges at 36.12 KiB and persists across larger payloads. At 36.12 KiB, the mean overlap is approximately 68% for Case 03, compared with approximately 42% for Case 01. For subsequent larger payloads, Case 01 generally exhibits lower overlap. Its mean overlap becomes slightly negative at 144.50 KiB and remains mostly below approximately 25% in the larger-payload region.

[2] The Open MPI configuration used `OMPI_MCA_btl_tcp_progress_thread=1`, `OMPI_MCA_mpi_yield_when_idle=0`, `OMPI_MCA_opal_progress_sched_yield=0`, and `OMPI_MCA_mpi_add_procs_cutoff=0`.

[3] The MPICH configurations use `MPIR_CVAR_ASYNC_PROGRESS=0` for Case 01 and `MPIR_CVAR_ASYNC_PROGRESS=1` for Case 03. In the tested MPICH build, enabling this variable creates one progress thread per MPI rank.

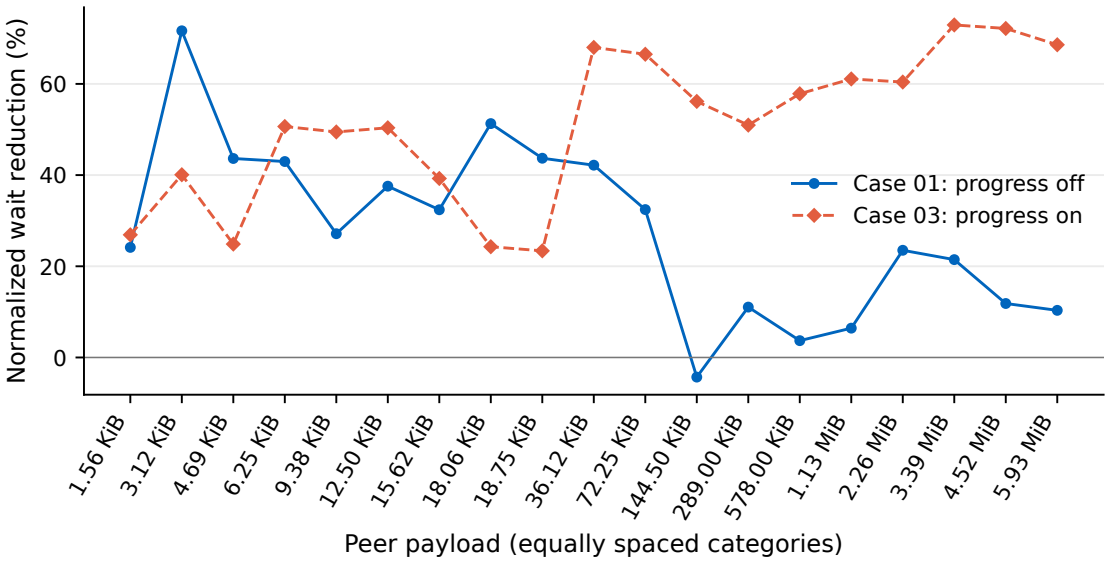


Figure 4.6.: MPICH TCP: measured overlap across 19 peer payloads with asynchronous progress disabled (Case 01, solid line with circles, 24 application threads) and enabled (Case 03, dashed line with diamonds, 23 application threads and one CPU reserved for progress). Both cases use `wait_all` and `seq/seq-seq` in the two-node, two-rank `cm4_inter` configuration.

In contrast, Case 03 maintains substantially higher overlap over the same range. From 36.12 KiB onward, its mean overlap mostly remains between approximately 50% and 70%, exceeding 70% at several of the largest payload points.

These results indicate that enabling MPICH asynchronous progress does not provide a consistent improvement for small payloads, but substantially improves communication–computation overlap for medium and large payloads.

Intel MPI

For Intel MPI, asynchronous progress is controlled through `I_MPI_ASYNC_PROGRESS`. Case 01 disables asynchronous progress, whereas Case 03 enables it and reserves one CPU for the progress thread.[4]

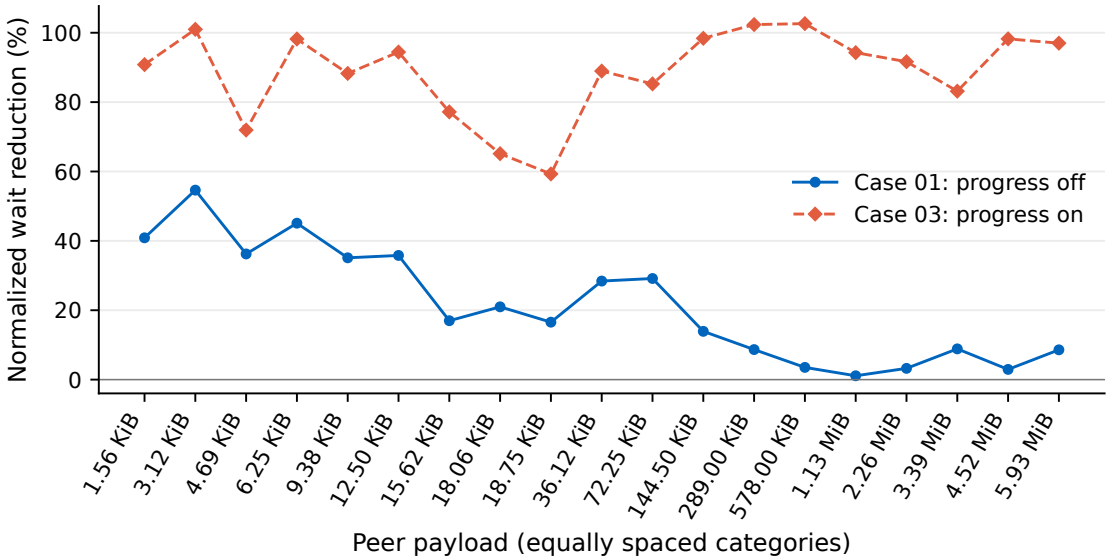


Figure 4.7.: Intel MPI TCP: measured overlap across 19 peer payloads with asynchronous progress disabled (Case 01, solid line with circles, 24 application threads) and enabled (Case 03, dashed line with diamonds, 23 application threads and one CPU reserved for progress). Both cases use `wait_all` and `seq/seq-seq` in the two-node, two-rank `cm4_inter` configuration.

As shown in Figures 4.7 and 4.8, the two Intel MPI transport configurations exhibit substantially different behavior when asynchronous progress is disabled.

[4]The Intel MPI configuration uses `I_MPI_ASYNC_PROGRESS=0` for Case 01 and `I_MPI_ASYNC_PROGRESS=1` for Case 03. For Case 03, `I_MPI_ASYNC_PROGRESS_THREADS=1` is used, with one CPU reserved for the progress thread.

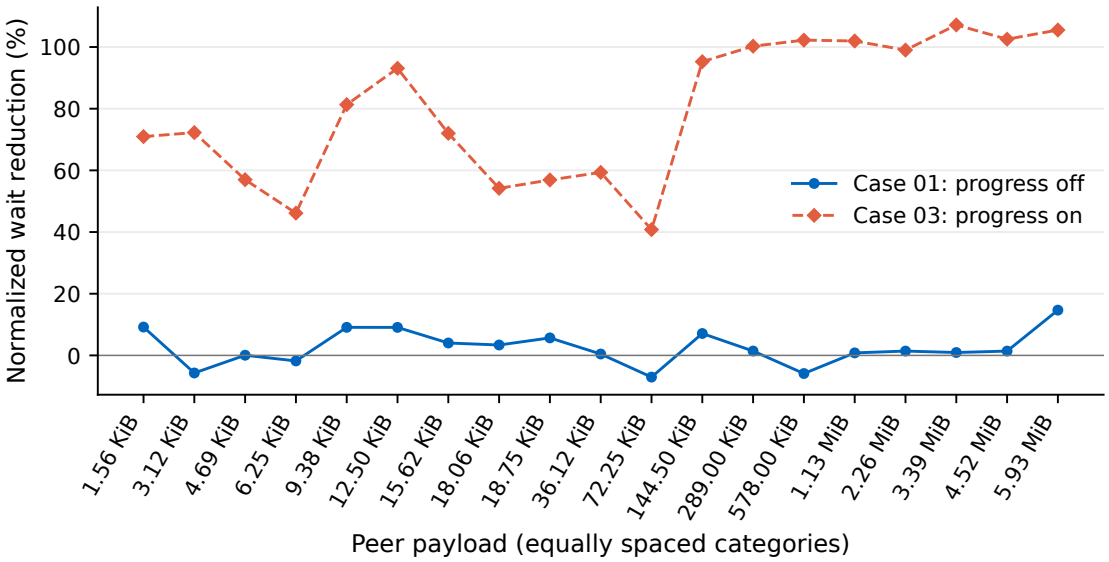


Figure 4.8.: Intel MPI verbs: measured overlap across 19 peer payloads with asynchronous progress disabled (Case 01, solid line with circles, 24 application threads) and enabled (Case 03, dashed line with diamonds, 23 application threads and one CPU reserved for progress). Both cases use `wait_all` and `seq/seq-seq` in the two-node, two-rank `cm4_inter` configuration.

For Intel MPI TCP, Case 01 shows moderate but irregular overlap in the small-payload region. At 1.56 KiB, the mean overlap is approximately 41%, and the values fluctuate between approximately 17% and 55% over much of the remaining small-payload range. As the payload increases, the overlap generally decreases. From approximately 578 KiB onward, it remains close to zero for most payloads.

Intel MPI verbs shows a markedly different Case 01 behavior. Across almost the complete payload sweep, the mean overlap remains close to zero, with most values lying approximately between -10% and 15%. Thus, without asynchronous progress, the verbs transport provides little sustained communication–computation overlap during the injected computation window.

Enabling asynchronous progress substantially changes the behavior of both transport paths. For TCP, Case 03 achieves considerably higher overlap across almost the entire payload range, with most values lying between approximately 60% and 100%. Unlike Case 01, no systematic collapse toward zero is observed for large payloads.

The effect is even more pronounced for verbs. Whereas Case 01 remains close to zero across most payloads, Case 03 exhibits substantial overlap throughout the sweep. For medium and large payloads, the measured overlap reaches approximately 95% to 105% for most points.

These results indicate that asynchronous progress has a substantial positive effect on Intel MPI communication–computation overlap for both TCP and verbs. The effect is particularly strong for the verbs transport, where enabling asynchronous progress changes R_{wait} from close to zero to approximately 100% for many medium and large payloads. At these points, the receive-wait reduction is approximately equal to the measured computation time; this does not by itself establish that all communication is hidden.

4.2.5. Impact of Completion Strategy: `wait_all` versus `test_all`

In Comb, `wait_all` calls `MPI_Waitall` to wait for all communication requests to complete, whereas `test_all` repeatedly calls `MPI_Testall` until all requests have completed. Both execute after the injected computation, so this comparison concerns the communication completion method rather than polling during computation.

This experiment examines whether replacing `wait_all` with `test_all` improves the measured communication–computation overlap when the injected computation time is approximately equal to the reference communication time. Case 01 is compared with Case 02 with asynchronous progress disabled, and Case 03 with Case 04 with progress enabled.

Under these test conditions, the two completion methods generally show similar measured overlap. Although differences occur at individual message sizes, `test_all`

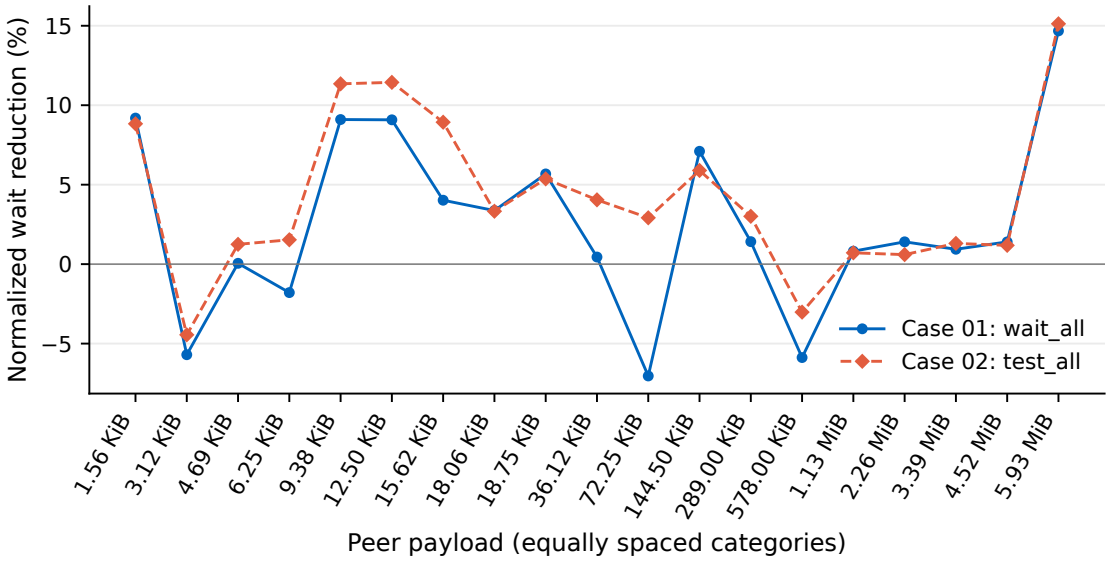


Figure 4.9.: Intel MPI verbs, Case 01 versus Case 02, with progress disabled and the `seq/seq-seq` policy. Solid blue circles denote `wait_all`; dashed orange diamonds denote `test_all`. Points are run means.

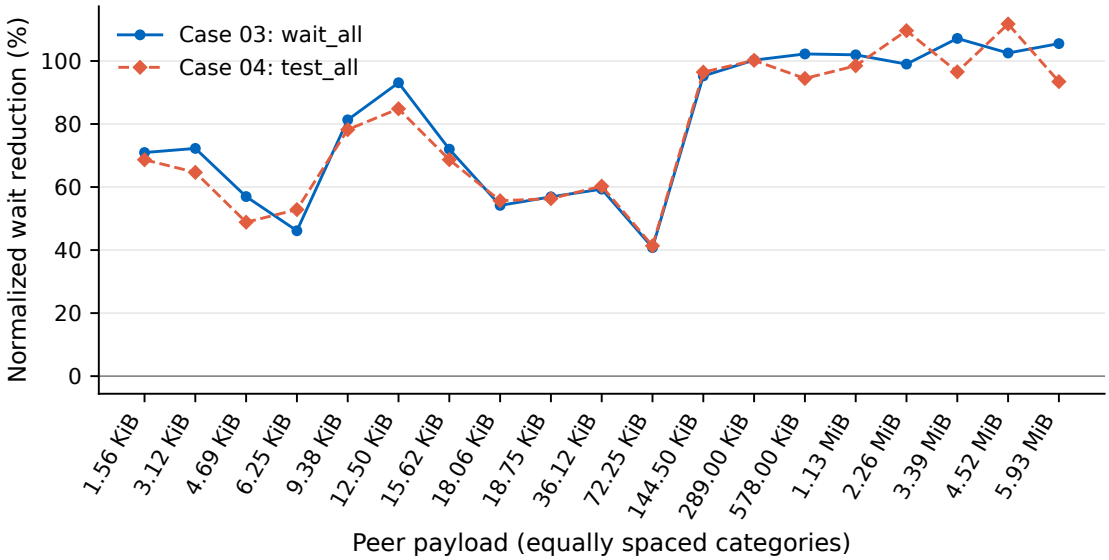


Figure 4.10.: Intel MPI verbs, Case 03 versus Case 04, with progress enabled and the `seq/seq-seq` policy. Solid blue circles denote `wait_all`; dashed orange diamonds denote `test_all`. Points are run means.

does not provide a consistent improvement. The Intel MPI verbs results are shown in Figure 4.9 and Figure 4.10; the Intel MPI TCP, Open MPI, and MPICH comparisons are provided in Appendix B.

4.3. Polling During Computation

Unlike the computation window described in Section 3.1.3, which contains no explicit MPI progress or completion calls, the polling configuration introduces two polling points at approximately equal intervals within the injected computation. At each point, outstanding receive and send requests are tested using `MPI_Testall`, after which computation continues without waiting for all requests to complete. Both cases retain `wait_all` for final communication completion.

4.3.1. Intel MPI verbs

Figures 4.11 and 4.12 compare measured overlap with and without polling at four peer payloads for Case 01 and Case 03, respectively. In the polling configuration, two `MPI_Testall` calls are inserted at approximately equal intervals within the injected computation. At each polling point, Comb tests the outstanding receive and send requests and then continues the computation without waiting for all requests to complete. Both cases use `wait_all` for final communication completion.

With the asynchronous progress thread disabled in Case 01, the selected polling results show higher mean overlap at all four payloads. This is consistent with explicit MPI calls during computation providing additional opportunities for communication progress in this configuration.

4.3.2. Open MPI TCP

Figures 4.13 and 4.14 show the results with asynchronous progress disabled and enabled, respectively.

For Open MPI TCP, inserting two polling points during computation does not provide a consistent improvement in measured overlap. At the two smaller payloads, the mean values with polling remain similar to the corresponding no-polling results. Although polling substantially increases the mean R_{wait} at 289.00 KiB when asynchronous progress is disabled, this behavior is not reproduced consistently at the other payloads or when asynchronous progress is enabled.

More importantly, the polling configurations exhibit substantially greater run-to-run variability at several payload sizes. In particular, at 18.06 KiB, 289.00 KiB, and 2.26 MiB,

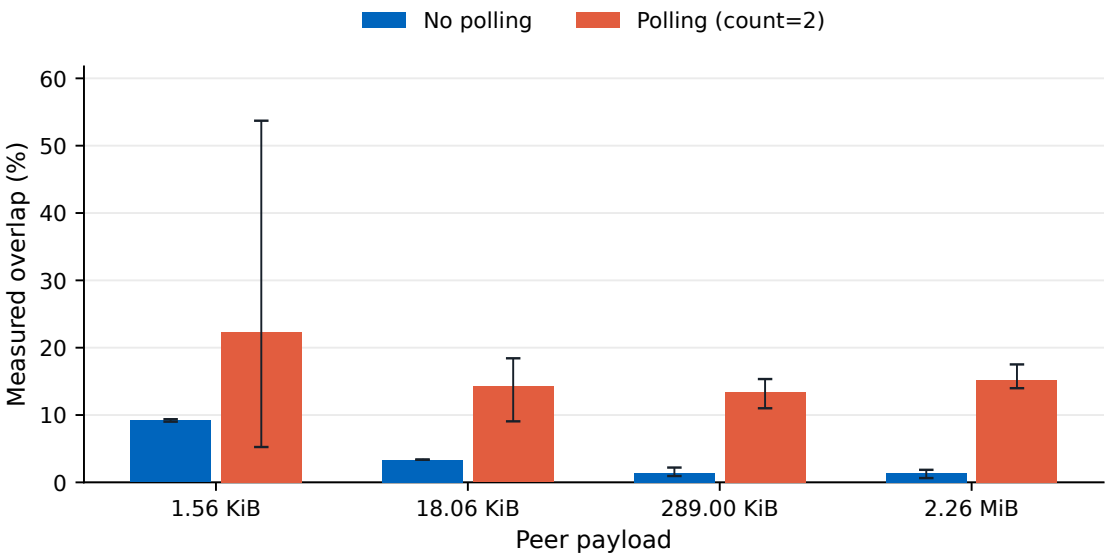


Figure 4.11.: Intel MPI verbs, Case 01: measured overlap with and without two polling points during computation. Asynchronous progress is disabled. Bars show run means, and error bars show the minimum–maximum range.

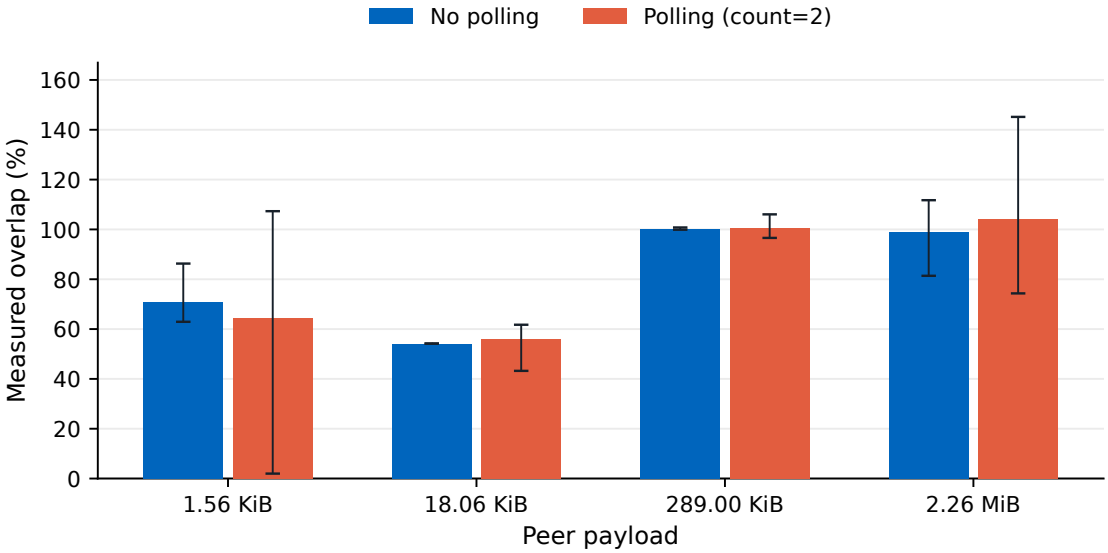


Figure 4.12.: Intel MPI verbs, Case 03: measured overlap with and without two polling points during computation. Asynchronous progress is enabled. Bars show run means, and error bars show the minimum–maximum range.

the minimum–maximum ranges are clearly wider than those of the corresponding no-polling measurements. This variability is especially pronounced at 2.26 MiB, where the polling results span both large positive and negative values.

Overall, under the configurations evaluated in this study, the results provide no evidence that explicit polling yields a stable and reproducible overlap benefit for Open MPI TCP. Instead, polling is associated with increased run-to-run variability in R_{wait}.

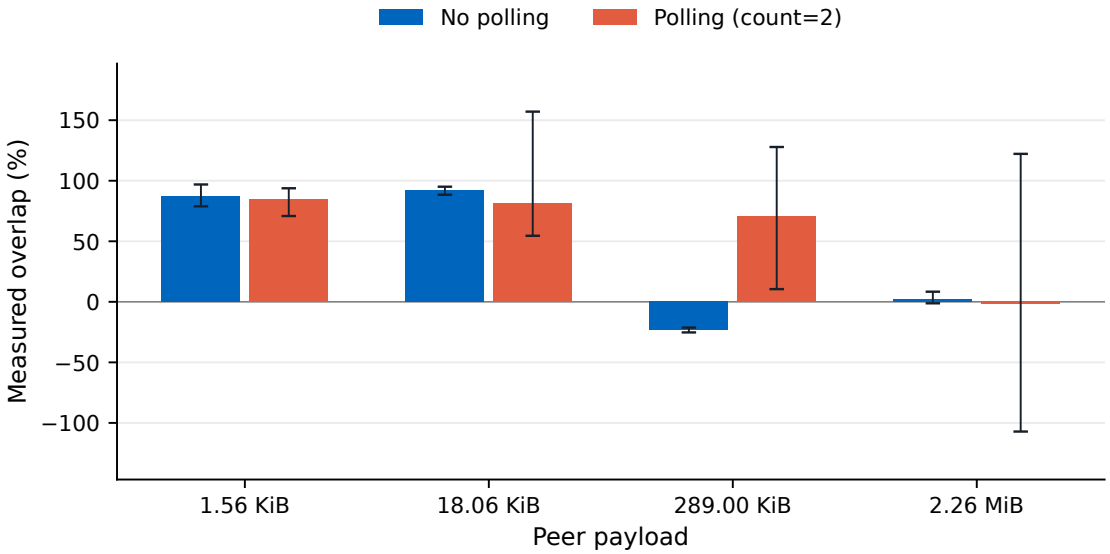


Figure 4.13.: Open MPI TCP, Case 01: measured overlap with and without two polling points during computation. Asynchronous progress is disabled. Bars show means across runs, and error bars show the minimum–maximum range.

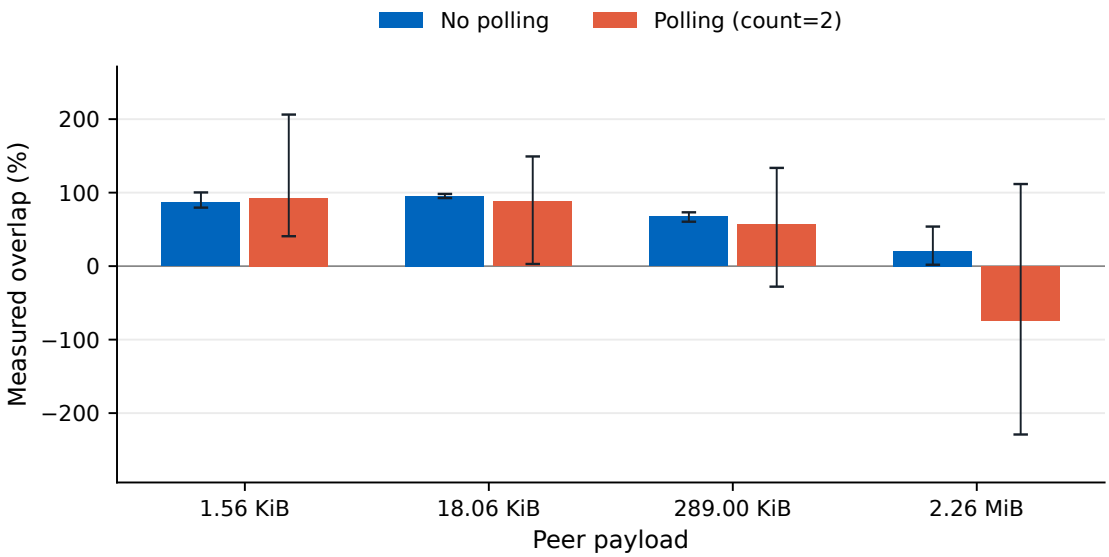


Figure 4.14.: Open MPI TCP, Case 03: measured overlap with and without two polling points during computation. Asynchronous progress is enabled. Bars show means across runs, and error bars show the minimum–maximum range.

5. Conclusion and Future Work

5.1. Summary

This thesis investigated communication–computation overlap through a benchmark-based evaluation on LRZ CoolMUC-4. Comb was extended with an adjustable synthetic computation window between communication initiation and completion, together with optional polling during computation. The experiments compared Open MPI over TCP, MPICH over TCP, and Intel MPI over verbs and TCP in a two-node, two-rank configuration across 19 peer payloads. The four evaluated cases distinguish asynchronous progress settings and final communication completion through `wait_all` or `test_all`. Additional Intel MPI verbs experiments examined polling during computation at four payloads.

The evaluation used changes in exposed receive completion time to quantify measured overlap. Comb's mock backend provided a reference for local framework and memory-handling costs. The resulting measurements characterize how the tested communication configurations behave during an injected computation window; they do not directly measure the execution time of a complete ocean model.

5.2. Conclusions

The experimental results show that different MPI implementations, transport configurations, and progress settings exhibit clearly different communication–computation overlap behavior, and that these differences vary with message size. Here, measured overlap refers to the normalized receive-wait reduction R_{wait} defined in section 3.5. In addition, simply using nonblocking communication does not guarantee effective communication–computation overlap. Without asynchronous progress, Intel MPI verbs exhibits little sustained measured overlap across most of the payload range. Intel MPI TCP shows moderate but irregular overlap for small payloads, while the overlap is close to zero for most large payloads. Enabling asynchronous progress substantially increases R_{wait} for both transport configurations. For Intel MPI verbs, the receive-wait reduction is approximately equal to the measured computation time at many medium and large payloads; this alone does not establish that all communica-

tion is hidden.

The effect of asynchronous progress also depends on the MPI implementation and message size. For Open MPI TCP, enabling the TCP BTL progress thread has relatively little effect for small payloads, but preserves substantially more overlap at larger payloads than the configuration without the thread. Nevertheless, the measured overlap still decreases as the large payloads increase. MPICH shows an inconsistent response to asynchronous progress for small payloads, while the progress-enabled configuration maintains substantially higher overlap in the medium- and large-payload range. Thus, even with progress enabled, the tested MPI implementations do not exhibit a common message-size dependence.

The comparison of communication completion methods in subsection 4.2.5 shows that, when the injected computation time is approximately equal to the reference communication time, replacing `MPI_Waitall` with a loop repeatedly calling `MPI_Testall` does not provide a consistent advantage. The two methods generally produce similar measured overlap, although differences remain at individual payloads. Since both methods execute after the computation has finished, this result only reflects differences in the final communication completion path and does not imply that polling during computation is ineffective or that the two methods are statistically equivalent.

Polling within the computation window produces a different response. In the Intel MPI verbs experiments described in section 4.3, when asynchronous progress is disabled, adding two polling points increases the run mean overlap at all four tested payloads. With asynchronous progress enabled, the additional benefit of polling is more limited and is not consistent across payloads.

This thesis experimentally characterizes the overlap of communication and computation in the evaluated environment, providing a basis for selecting and testing progress strategies for halo-exchange workloads.

5.3. Limitations and Future Work

One major limitation of this study is the limited number of independent repetitions. Although each configuration was evaluated using multiple independent runs to observe run-to-run variability, the measurements for some message sizes, especially in the small-message region, exhibited substantial variability. Therefore, the current experiments are sufficient to reveal the main trends, but the statistical stability of small differences at individual message sizes and of highly variable data points remains limited. Future work could increase the number of independent runs and use confidence intervals or other statistical measures to determine more systematically whether the observed differences are stable.

A second limitation arises from the timing methodology itself. Timers were inserted around the communication initiation, computation window, and communication completion phases according to the execution structure of Comb, while the Mock backend was used to correct for local framework and memory-handling overhead as far as possible. However, these timing intervals do not directly correspond to the actual communication phases inside the MPI implementation. Internal MPI protocol processing, data progress, and some local processing may cross the timing boundaries defined in this study. Therefore, the measured communication times and overlap metrics should be interpreted as approximate measures based on program-observable timing intervals rather than as direct measurements of the internal network progress of MPI. Future work could combine the benchmark with lower-level tracing, MPI profiling, or hardware performance counters to further validate where and when communication actually occurs.

Third, the experimental parameters in this study were primarily chosen to construct a controlled benchmark environment and do not completely match the runtime parameters of a specific full ocean model. The message payload sizes, computation-window durations, computation-to-communication ratios, MPI-rank topology, and synthetic computation workload were deliberately controlled in order to study the effects of individual factors separately. Therefore, the overlap behavior observed in this study cannot be directly equated with application performance in a real ocean model. Future work should extract representative message sizes, computation times, and communication patterns from actual ocean-model workloads and validate the results using realistic computation kernels or complete applications.

In addition, the main experiments in this thesis are limited to a small set of MPI implementations, transport paths, and node configurations on LRZ CoolMUC-4. Extending the evaluation to more nodes, multiple MPI ranks per node, and different CPU and progress-thread resource allocations would allow the effects of shared-node resource contention and more complex communication topologies on communication progress to be investigated. The explicit polling experiments within the computation window are also currently limited to Intel MPI verbs, four payload sizes, and a fixed number of polling points. Future experiments could systematically vary the polling frequency and placement to determine under which conditions the additional communication progress provided by explicit polling is sufficient to offset the overhead of entering MPI during computation.

Finally, representative ocean-model kernels and application-level experiments are needed to verify whether the communication–computation overlap observed in Comb persists under realistic memory-access, threading, and numerical computation patterns, and whether this overlap can actually reduce the total execution time of a complete simulation.

A. Reproducibility Information

A.1. Experiment Inventory

The repository accompanying this thesis provides the experimental results and the web-based experiment management and visualization tool introduced in section 1.1. The tool supports experiment configuration, job-script generation, run management, and visualization of measured results. The repository is available at:

`https://github.com/jintaolutum-dev/mpi-async-overlap-stud`

A.2. Message-Size Overlap Figure Set

The following figures compare asynchronous progress disabled (Case 01) and enabled (Case 03) across the complete message-size sweep for each MPI implementation and transport configuration. All figures use the `cm4_inter` two-node, two-rank configuration and the MPI `seq/seq-seq` policy. Each curve shows the mean measured overlap across the experimental runs.

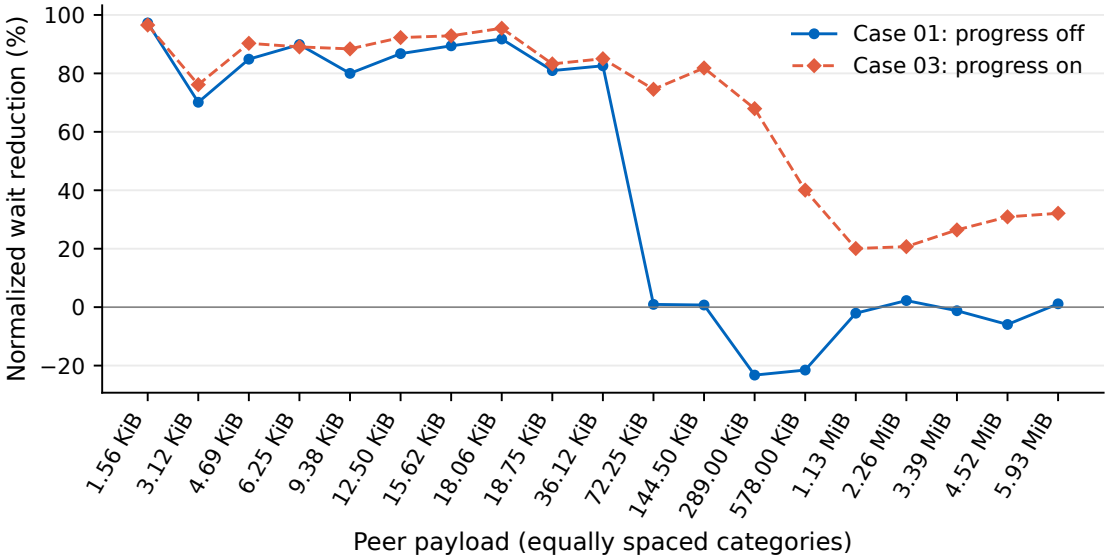


Figure A.1.: Open MPI TCP: measured overlap across 19 peer payloads with asynchronous progress disabled (Case 01, solid line with circles, 24 application threads) and enabled (Case 03, dashed line with diamonds, 23 application threads and one CPU reserved for progress). Both cases use `wait_all` and `seq/seq-seq` in the two-node, two-rank `cm4_inter` configuration.

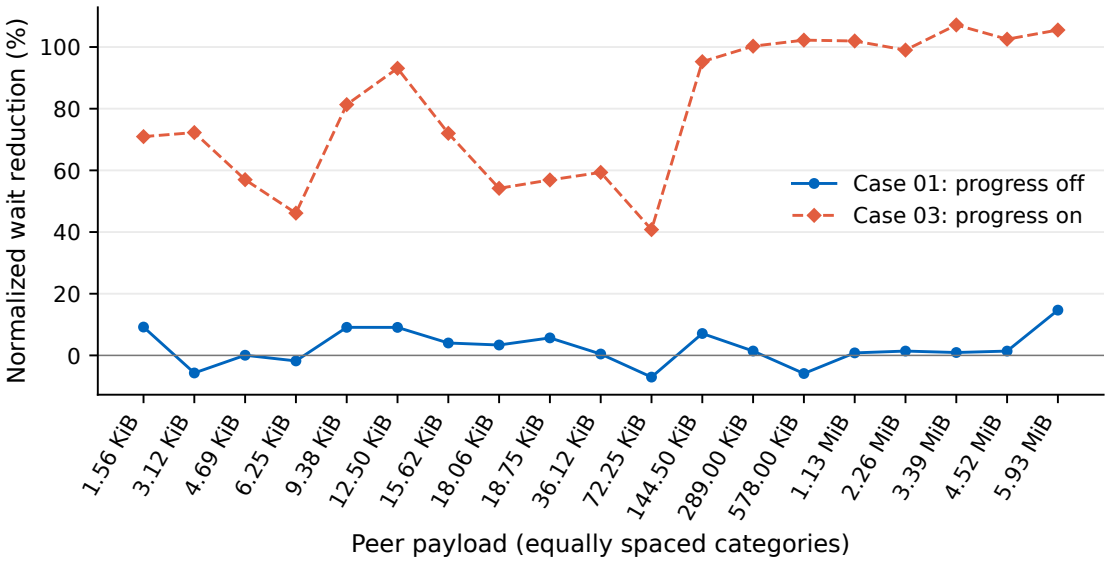


Figure A.2.: Intel MPI verbs: measured overlap across 19 peer payloads with asynchronous progress disabled (Case 01, solid line with circles, 24 application threads) and enabled (Case 03, dashed line with diamonds, 23 application threads and one CPU reserved for progress). Both cases use `wait_all` and `seq/seq-seq` in the two-node, two-rank `cm4_inter` configuration.

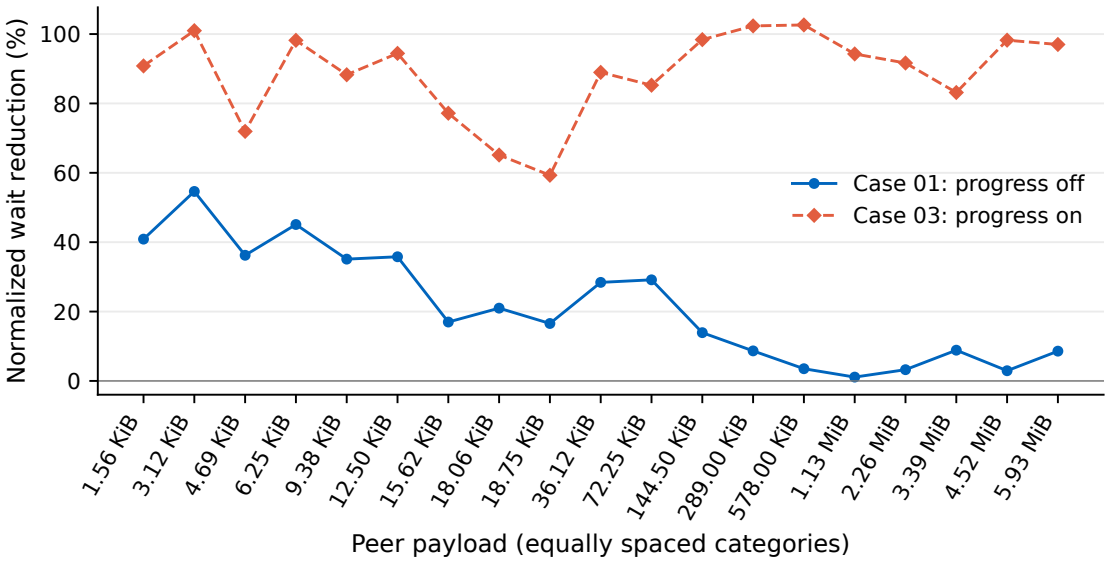


Figure A.3.: Intel MPI TCP: measured overlap across 19 peer payloads with asynchronous progress disabled (Case 01, solid line with circles, 24 application threads) and enabled (Case 03, dashed line with diamonds, 23 application threads and one CPU reserved for progress). Both cases use `wait_all` and `seq/seq-seq` in the two-node, two-rank `cm4_inter` configuration.

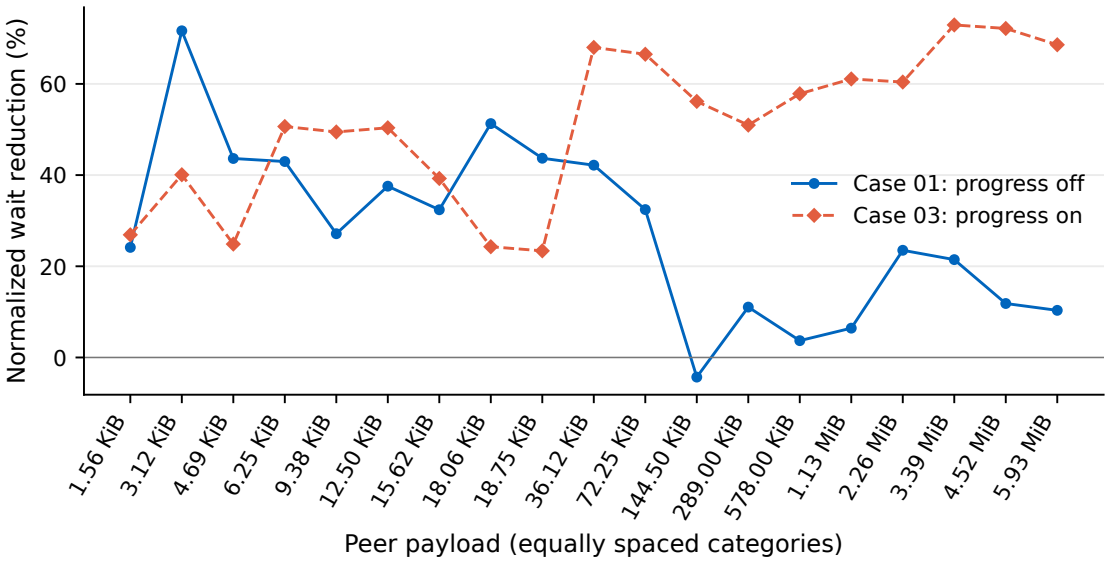


Figure A.4.: MPICH TCP: measured overlap across 19 peer payloads with asynchronous progress disabled (Case 01, solid line with circles, 24 application threads) and enabled (Case 03, dashed line with diamonds, 23 application threads and one CPU reserved for progress). Both cases use `wait_all` and `seq/seq-seq` in the two-node, two-rank `cm4_inter` configuration.

B. Additional Completion-Strategy Comparisons

These figures complement the Intel verbs comparisons in subsection 4.2.5. All use the two-node, two-rank `cm4_inter` configuration and the MPI `seq/seq-seq` policy. Solid circles denote `wait_all`, and dashed orange diamonds denote `test_all`. Case 01/02 disables progress; Case 03/04 enables the implementation-specific progress configuration. Points show the mean normalized wait reduction across experimental runs. These plots show means, not confidence intervals, and do not establish equivalence of the completion methods.

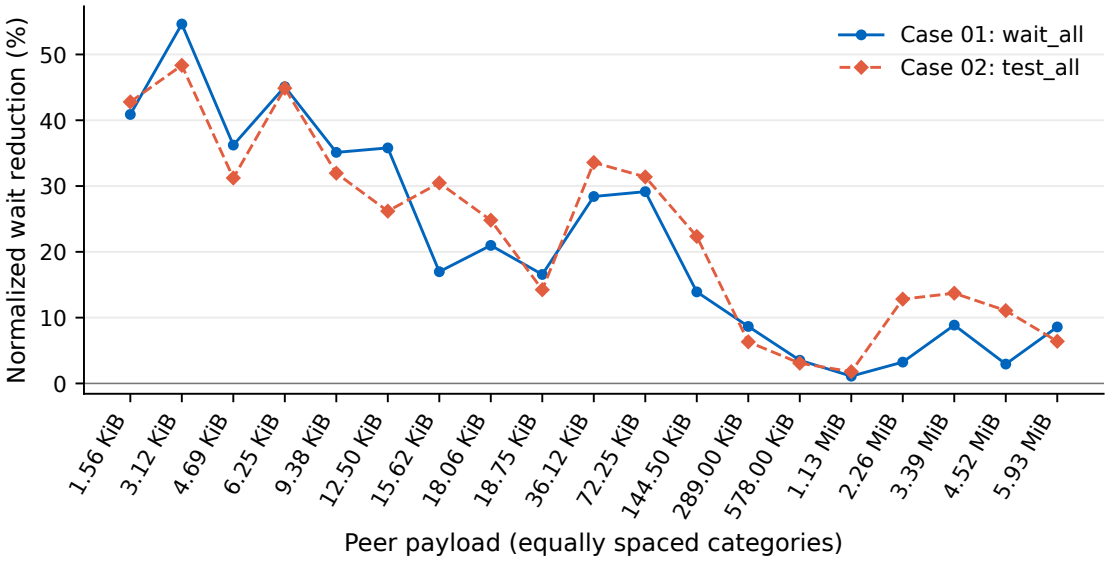


Figure B.1.: Intel MPI TCP: Case 01 versus Case 02, progress disabled.

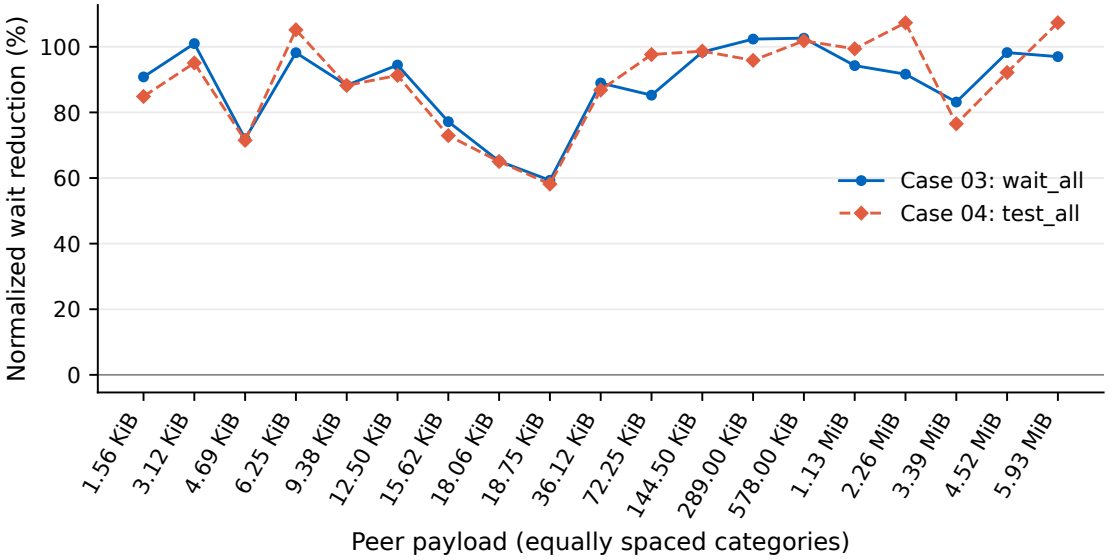


Figure B.2.: Intel MPI TCP: Case 03 versus Case 04, progress enabled.

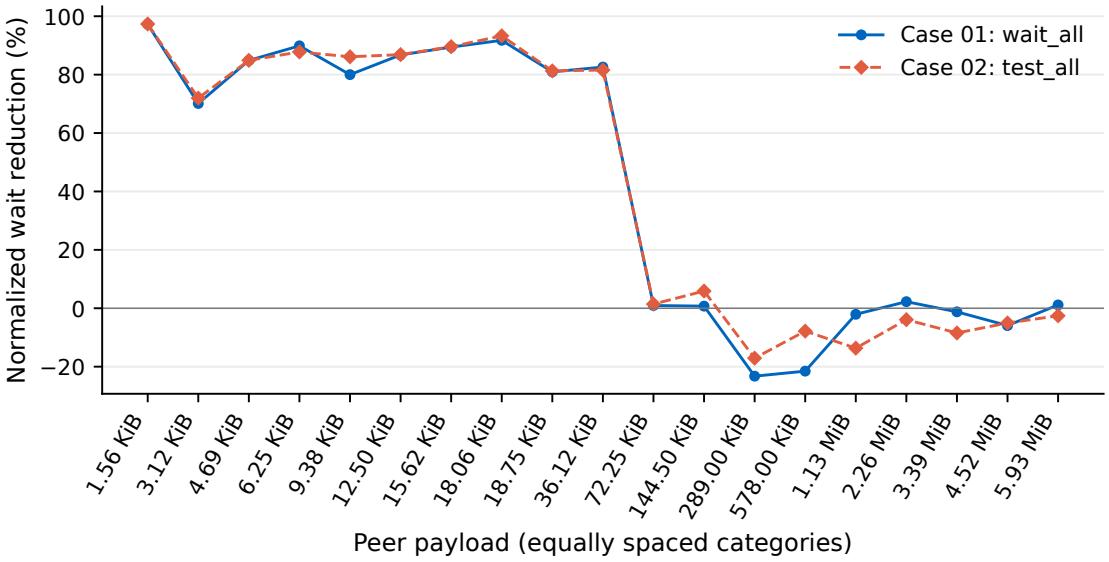


Figure B.3.: Open MPI TCP: Case 01 versus Case 02, progress disabled.

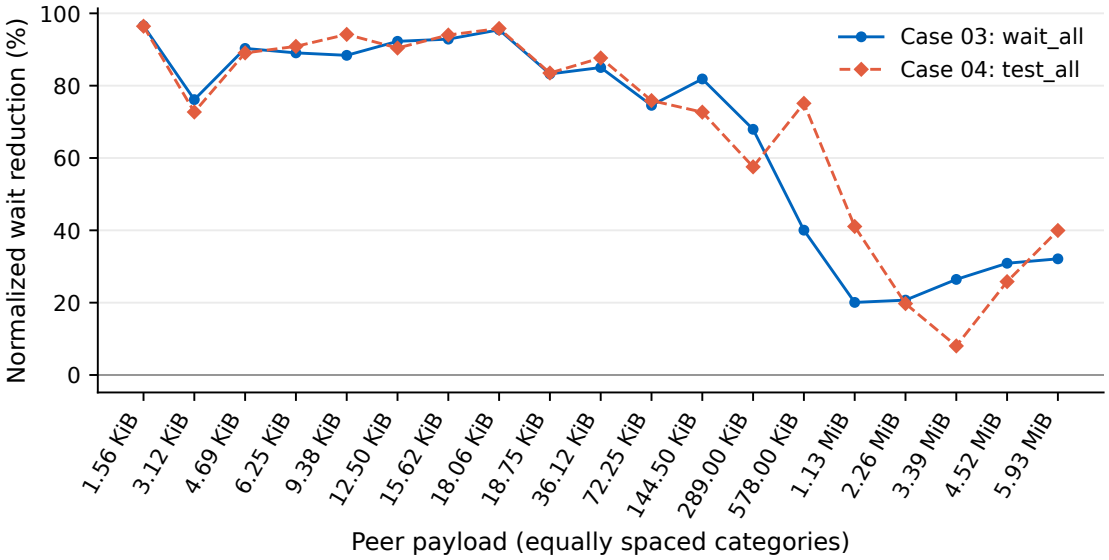


Figure B.4.: Open MPI TCP: Case 03 versus Case 04, TCP progress enabled.

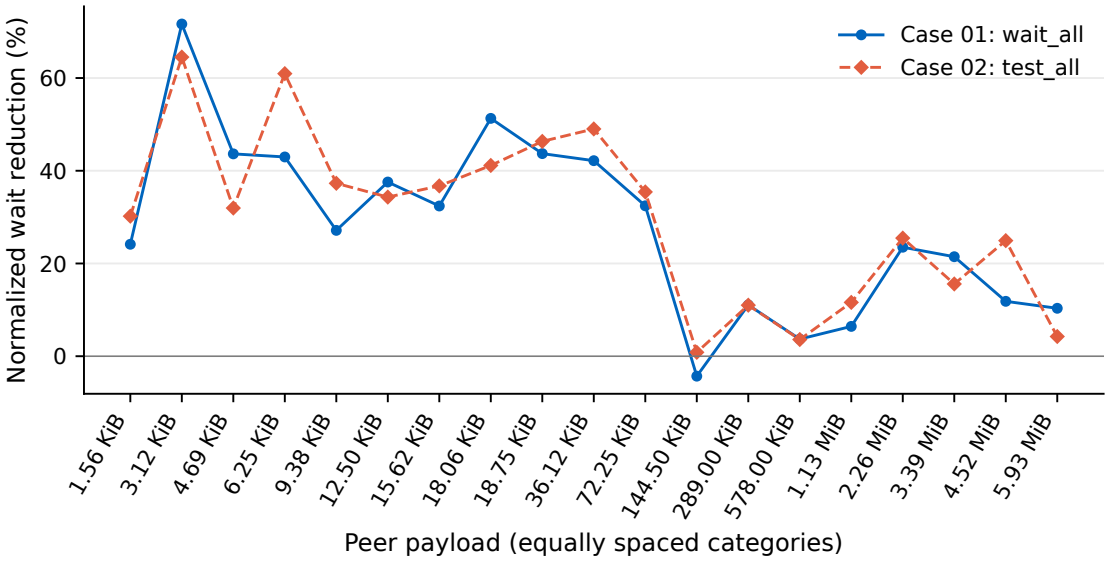


Figure B.5.: MPICH TCP: Case 01 versus Case 02, progress disabled.

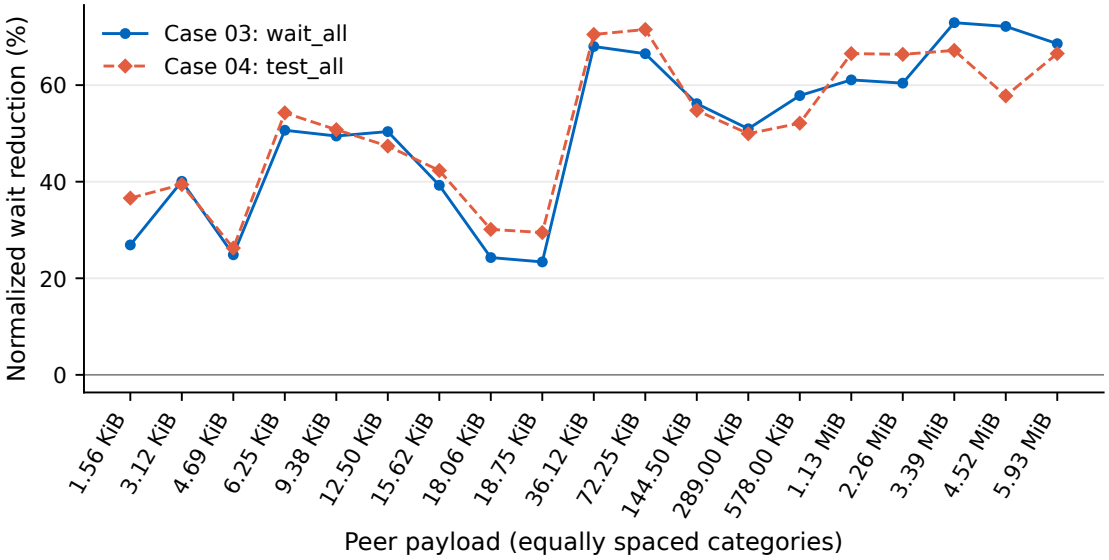


Figure B.6.: MPICH TCP: Case 03 versus Case 04, progress enabled.

List of Figures

2.1. Schematic comparison of blocking and nonblocking MPI point-to-point communication. Nonblocking operations separate initiation from completion, allowing independent computation to potentially overlap with communication. Actual overlap depends on communication progress during computation. The timelines are illustrative and not to scale. . . . 5

3.1. Phases executed by each rank in one timed Comb communication cycle. Gray boxes denote local mesh operations, blue boxes MPI communication phases, and the green box the injected computation window. Receive and send completion use either Waitall or a Testall loop; the timer names remain `wait-recv` and `wait-send` in both cases. The return arrow connects successive cycles. 20
3.2. Matched MPI-only and Mock-only cost measurement. 21
3.3. Normalized receive-wait reduction. 22

4.1. Open MPI overlap across message sizes. 29
4.2. MPICH overlap across message sizes. 30
4.3. Intel MPI verbs overlap across message sizes. 32
4.4. Intel MPI TCP overlap across message sizes. 32
4.5. Open MPI TCP asynchronous progress off versus on. 33
4.6. MPICH TCP asynchronous progress off versus on. 35
4.7. Intel MPI TCP asynchronous progress off versus on. 36
4.8. Intel MPI verbs asynchronous progress off versus on. 37
4.9. Intel verbs completion comparison with progress disabled. 39
4.10. Intel verbs completion comparison with progress enabled. 39
4.11. Intel MPI verbs polling with asynchronous progress disabled. 41
4.12. Intel MPI verbs polling with asynchronous progress enabled. 41
4.13. Open MPI polling with asynchronous progress disabled. 42
4.14. Open MPI polling with asynchronous progress enabled. 43

A.1. Open MPI TCP asynchronous progress off versus on. 48
A.2. Intel MPI verbs asynchronous progress off versus on. 49
A.3. Intel MPI TCP asynchronous progress off versus on. 50

A.4. MPICH TCP asynchronous progress off versus on. 51

B.1. Intel MPI TCP: Case 01 versus Case 02, progress disabled. 53
B.2. Intel MPI TCP: Case 03 versus Case 04, progress enabled. 53
B.3. Open MPI TCP: Case 01 versus Case 02, progress disabled. 54
B.4. Open MPI TCP: Case 03 versus Case 04, TCP progress enabled. 54
B.5. MPICH TCP: Case 01 versus Case 02, progress disabled. 55
B.6. MPICH TCP: Case 03 versus Case 04, progress enabled. 55

List of Tables

2.1. Conceptual communication-stack layers of the evaluated MPI implementations. 8

3.1. MPI implementations and communication configurations used in the experiments. 14
3.2. Per-rank completion and CPU-resource configurations. Intel columns apply to both verbs and TCP. Open MPI reports reserved CPU IDs, not a configured progress-thread count. 17
3.3. Aggregated inter-node partner payloads for the complete message-size sweep in the two-rank reference topology. 19

4.1. Ten-run variability for Intel MPI TCP. 28
4.2. Two-rank Mock-corrected communication cost. 28

Bibliography

[1] R. H. Castain and J. M. Squyres. *The ABCs of Open MPI: Decoding the Alphabet Soup of the Modern HPC Ecosystem (Part 2)*. Presentation, July 8, 2020; MPI frameworks and network-stack selection. URL: https://www.open-mpi.org/video/general/easybuild_tech_talks_01_OpenMPI_part2_20200708.pdf (visited on 09/13/2026).

[2] A. Denis, J. Jaeger, E. Jeannot, M. Pérache, and H. Taboada. "Study on progress threads placement and dedicated cores for overlapping MPI nonblocking collectives on manycore processor." In: *The International Journal of High Performance Computing Applications* 33.6 (2019), pp. 1240–1254. DOI: 10.1177/1094342019860184.

[3] A. Denis and F. Trahay. "MPI Overlap: Benchmark and Analysis." In: *2016 45th International Conference on Parallel Processing (ICPP)*. IEEE, 2016, pp. 258–267. DOI: 10.1109/ICPP.2016.37.

[4] Intel Corporation. *Asynchronous Progress Control*. Documentation version 2021.11. Intel MPI Library Developer Guide for Linux OS. URL: https://www.intel.com/content/www/us/en/docs/mpi-library/developer-guide-linux/2021-11/asynchronous-progress-control.html (visited on 09/12/2026).

[5] Intel Corporation. *Communication Fabrics Control*. Intel MPI Library Developer Reference, version 2021.15. URL: https://www.intel.com/content/www/us/en/docs/mpi-library/developer-reference-linux/2021-15/communication-fabrics-control.html (visited on 09/13/2026).

[6] Intel Corporation. *Environment Variables for Asynchronous Progress Control*. Documentation version 2021.15. Intel MPI Library Developer Reference for Linux OS. URL: https://www.intel.com/content/www/us/en/docs/mpi-library/developer-reference-linux/2021-15/environment-variables-for-async-progress-control.html.

[7] Intel Corporation. *OFI Providers Support*. Intel MPI Library Developer Guide, version 2021.15. URL: https://www.intel.com/content/www/us/en/docs/mpi-library/developer-guide-linux/2021-15/ofi-providers-support.html (visited on 09/13/2026).

[8] Lawrence Livermore National Laboratory. *Comb: Communication Performance Benchmarking Tool*. Comb version 0.3.1. URL: https://github.com/LLNL/Comb (visited on 09/12/2026).

[9] Leibniz-Rechenzentrum. *CoolMUC Cheat Sheet*. Partition selection, including the cm4_inter interactive partition. URL: https://doku.lrz.de/files/1646760668/1898711114/3/1755197396847/CoolMUC-Cheat-Sheet.pdf (visited on 09/14/2026).

[10] Leibniz-Rechenzentrum. *Linux Cluster*. CoolMUC-4 hardware and interconnect overview. URL: https://www.lrz.de/service-center/alle-lrz-services/high-performance-computing/linux-cluster (visited on 09/14/2026).

[11] Message Passing Interface Forum. *MPI: A Message-Passing Interface Standard, Version 5.0*. June 5, 2025.

[12] MITgcm Team. *MITgcm Documentation: Communication Primitives*. Section 4.2.8. URL: https://mitgcm.org/sealion/online_documents/node160.html (visited on 09/13/2026).

[13] A. Mittermair, M. Schulz, J. Remy, H. Brunie, and M. Schreiber. *Automatic Distributed Parallelization and Communication Optimization for Stencils on Structured Grids*. Preprint. 2025. HAL: hal-05429086. URL: https://hal.science/hal-05429086v1 (visited on 09/01/2026).

[14] MPI Performance Guidelines Contributors. *Ensuring Progress for MPI Nonblocking Operations*. URL: https://mpi-performance-guidelines.github.io/progress.html (visited on 09/13/2026).

[15] MPICH Development Team. *MPICH Asynchronous Progress: Configuration and Implementation*. Official source documentation for MPIR_CVAR_ASYNC_PROGRESS. Argonne National Laboratory. URL: https://github.com/pmodels/mpich/blob/v4.2.2/src/mpi/init/init_async.c (visited on 09/12/2026).

[16] MPICH Development Team. *MPICH Developer Guide*. CH4, shared-memory modules, and OFI/UCX network modules. URL: https://github.com/pmodels/mpich/blob/main/doc/wiki/developer_guide.md (visited on 09/13/2026).

[17] Open MPI Development Team. *FAQ: Tuning the Run-Time Characteristics of MPI TCP Communications*. Documentation for Open MPI 4.x and earlier. URL: https://www.open-mpi.org/faq/?category=tcp (visited on 09/13/2026).

[18] Open MPI Development Team. *Open MPI 4.1.8 TCP BTL Component: Progress-Thread Configuration and Initialization*. Official source code, version 4.1.8. URL: https://github.com/open-mpi/ompi/blob/v4.1.8/opal/mca/btl/tcp/btl_tcp_component.c (visited on 09/12/2026).

[19] A. Ruhela, H. Subramoni, S. Chakraborty, M. Bayatpour, P. Kousha, and D. K. Panda. "Efficient design for MPI asynchronous progress without dedicated resources." In: *Parallel Computing* 85 (2019), pp. 13–26. DOI: 10.1016/j.parco.2019.03.003.

[20] D. Schafer, T. Hines, E. D. Suggs, M. Rüfenacht, and A. Skjellum. "Overlapping Communication and Computation with ExaMPI's Strong Progress and Modern C++ Design." In: *2021 Workshop on Exascale MPI (ExaMPI)*. IEEE, 2021. DOI: 10.1109/EXAMPI54564.2021.00008.

[21] H. Zhou, R. Latham, K. Raffenetti, Y. Guo, and R. Thakur. *MPI Progress For All*. Preprint, version 2. 2024. DOI: 10.48550/arXiv.2405.13807. arXiv: 2405.13807. URL: https://arxiv.org/abs/2405.13807.

[22] H. Zhou, K. Raffenetti, Y. Guo, T. Gillis, R. Latham, and R. Thakur. *Designing and Prototyping Extensions to MPI in MPICH*. Preprint. 2024. DOI: 10.48550/arXiv.2402.12274. arXiv: 2402.12274. URL: https://arxiv.org/abs/2402.12274.